

Raspberry Pi 5와 ESP32 기반 상태 검증 엣지 제어 시스템 구현 및 통신·자원·병목 분석 보고서

Edge IoT Control System:
Implementation and Performance Analysis of a Raspberry Pi-ESP32
Edge Control Path

항목	내용
학교	아주대학교
학과	소프트웨어학과
이름	오재영
프로젝트명	Edge IoT & Control System
보고서 주제	Raspberry Pi 5와 ESP32 기반 상태 검증 엣지 제어 시스템 구현 및 통신·자원·병목 분석
진행 기간	2026.07 ~ 2026.09
저장소	GitHub: edge-iot-control-system

초록.....	4
1. 프로젝트 개요.....	5
1.1 프로젝트 배경.....	5
1.1.1 초기 프로젝트 동기.....	6
1.1.2 엣지 제어 시스템과 전체 제어 경로.....	6
1.1.3 기능 구현에서 성능 측정 및 분석으로의 확장.....	7
1.2 문제 정의.....	7
1.3 프로젝트 목표.....	8
1.3.1 실제 하드웨어 제어 시스템 구현.....	8
1.3.2 상태 검증 기반 조건부 제어.....	9
1.3.3 통신 방식에 따른 성능 비교.....	9
1.3.4 제어 경로의 지연 구성 분석.....	10
1.3.5 임베디드 및 엣지 장치의 자원 상태 관찰.....	10
1.4 분석 질문.....	10
1.5 프로젝트 범위.....	11
1.6 본 프로젝트에서 직접 측정하지 않은 범위.....	11
2. 시스템 구성.....	12
2.1 전체 하드웨어 구성.....	12
2.2 Raspberry Pi 5 엣지 제어 서버.....	14
2.3 ESP32 하드웨어 제어 노드.....	14
2.4 서보모터 및 외부 전원 구성.....	15
2.5 PC 전원 상태 검증을 위한 LDR 및 Ping.....	15
2.6 소프트웨어 환경.....	17
3. 시스템 설계.....	18
3.1 전체 시스템 구조.....	18
3.2 Light Switch Node 제어 경로.....	19
3.3 PC Power Node 제어 경로.....	20
3.4 상태 검증 기반 조건부 제어.....	21
3.5 HTTP 기반 통신 구조.....	21
3.6 MQTT 기반 Benchmark 구조.....	22
3.7 제어 경로 지연시간 분해 모델.....	23
3.8 실제 측정 범위와 개념적 모델의 구분.....	24
3.9 의도된 제어 지연과 시스템 오버헤드의 구분.....	26
3.10 메모리 및 자원 분석 기준.....	26
3.11 실험 데이터 관리 구조.....	27
4. 구현 과정.....	28

4.1 Raspberry Pi 초기 환경 구성.....	28
4.2 Flask 기반 Dashboard 및 중앙 제어 서버 구현.....	28
4.3 ESP32 Wi-Fi 연결 및 HTTP Server 구현.....	29
4.4 Light Switch Node 구현.....	29
4.5 PC Power Node 구현.....	30
4.6 Ping과 LDR을 결합한 상태 판단 구현.....	30
4.7 서보모터 반복 동작 및 안정성 확인.....	31
4.8 벽면 스위치 제어 실패와 물리적 문제 분석.....	31
4.9 MG996R 교체 및 제어 각도 보정.....	31
4.10 MQTT Benchmark 구조 추가.....	33
4.11 성능 측정을 위한 Instrumentation 추가.....	33
4.12 실험 데이터 및 분석 구조 정리.....	33
5. 실험 설계 및 측정 방법.....	34
5.1 실험 설계 원칙.....	34
5.2 기존 HTTP Baseline.....	35
5.3 Wi-Fi Sleep Diagnostic.....	36
5.4 최종 HTTP/MQTT Protocol Benchmark.....	36
5.5 ESP32 내부 Processing Time 측정.....	38
5.6 ESP32 Heap 변화 측정.....	38
5.7 Raspberry Pi 자원 사용량 측정.....	38
5.8 Light Switch End-to-End Latency 측정.....	39
5.9 액추에이터 물리 제어 성공률 측정.....	40
5.10 측정 지표 및 분석 방법.....	41
6. 기존 구현 및 초기 실험 결과.....	41
6.1 Raspberry Pi 초기 상태.....	41
6.2 기존 HTTP 통신 지연시간.....	41
6.3 ESP32 초기 Heap 변화.....	42
6.4 서보모터 반복 동작 및 온도 변화.....	43
6.5 MG90S 벽면 스위치 제어 결과.....	44
6.6 MG996R 교체 후 제어 결과.....	44
6.7 LDR 및 Ping 기반 PC 상태 판단 결과.....	45
7. 추가 성능 실험 결과.....	45
7.1 Wi-Fi Sleep과 MQTT Latency.....	45
7.2 HTTP 및 MQTT Application RTT 비교.....	46
7.3 Latency Distribution 및 Run별 재현성.....	48
7.4 ESP32 내부 Processing Time 분석.....	50

7.5 Application RTT의 구성 및 Remainder 분석.....	51
7.6 ESP32 Heap 변화 분석.....	52
7.7 Raspberry Pi 자원 사용량 분석.....	53
7.8 Light Switch End-to-End Latency.....	55
7.9 종합 Bottleneck 분석.....	57
8. 문제 해결 과정 및 결과 해석.....	57
8.1 통신 성공과 물리 제어 성공의 분리.....	57
8.2 서보모터 Torque·Mounting·Angle 문제.....	58
8.3 예상보다 높았던 MQTT Latency 분석.....	58
8.4 HTTP Keep-Alive 실험과 제외 판단.....	59
8.5 Raspberry Pi Resource 실험 재설계.....	59
8.6 ESP32 Heap 분석 방법 수정.....	60
8.7 의도된 Delay와 성능 병목의 구분.....	60
8.8 측정 데이터 정합성 및 Raw Data 관리.....	60
8.9 결과 해석 시 주의점.....	61
8.10 실험의 한계.....	61
9. 종합 결론.....	62
9.1 시스템 구현 결과.....	62
9.2 분석 질문별 결론.....	62
9.3 프로젝트를 통해 확인한 시스템 관점.....	63
9.4 프로젝트의 한계.....	64
9.5 이후 관심 방향.....	64
10. 셀프 피드백.....	64
10.1 프로젝트를 통해 학습한 내용.....	64
10.2 초기 프로젝트 진행 방식의 아쉬운 점.....	65
10.3 기능 구현에서 측정 및 분석으로의 관점 변화.....	65
11. 참고자료.....	66
부록 A. 주요 실험 데이터 및 Figure.....	66
A.1 Final Protocol Raw Data.....	66
A.2 Raspberry Pi Resource Data.....	67
A.3 Light End-to-End Data.....	67
A.4 주요 Figure.....	67
부록 B. 실험 재현을 위한 환경 기록.....	68

초록

본 프로젝트는 Raspberry Pi 5와 ESP32를 이용하여 실제 물리 장치를 원격으로 제어하는 시스템을 구현하고, 구현 과정에서 발생한 문제를 개선한 뒤 통신 지연시간, ESP32 내부 처리시간과 Heap 변화, Raspberry Pi 자원 사용량, 실제 물리 구동 지연 및 제어 신뢰성을 측정·분석한 개인 프로젝트임.

초기 프로젝트에서는 Raspberry Pi를 중앙 엣지 제어 서버로 구성하고 ESP32를 무선 하드웨어 제어 노드로 사용함. Light Switch Node에서는 Servo Motor를 이용하여 기존 벽면 전등 스위치를 물리적으로 제어하였으며, PC Power Node에서는 Raspberry Pi의 Ping 결과와 ESP32에 연결한 LDR Sensor의 측정 결과를 결합하여 PC의 현재 전원 상태를 확인한 뒤 필요한 경우에만 실제 Power Button을 누르도록 구성함.

프로젝트는 처음부터 성능 분석을 목적으로 시작한 것은 아니었음. 실제 생활에서 사용할 수 있는 Raspberry Pi-ESP32 기반 장치를 직접 구현하는 것이 초기 목표였으며, 기능 구현 이후 실제 환경에서 나타나는 문제를 해결하는 과정에서 프로젝트의 범위가 자연스럽게 확장됨. 특히 Software Command가 정상적으로 실행되더라도 실제 물리 제어가 실패하는 현상과, 통신 방식 및 장치 설정에 따라 응답시간이 예상과 다르게 나타나는 현상을 경험하면서 단순히 기능이 동작하는지를 확인하는 것에서 나아가 시스템의 동작 특성을 직접 측정하고 원인을 분석하는 방향으로 프로젝트를 발전시킴.

실제 벽면 스위치 제어에서는 초기 MG90S Servo를 이용한 20회의 시험에서 7회만 성공하여 35.0%의 성공률을 기록함. Raspberry Pi-ESP32 통신과 Servo Command는 정상적으로 수행되고 있었으므로 실패 원인을 물리 제어 계층에서 추가로 확인함. 이후 MG996R Servo로 교체하고 장착 구조를 보강하였으며, Servo Angle을 5° 또는 10° 단위로 조정함. 최종적으로 REST 90°, ON 50°, OFF 140°로 설정하였고 ON 20회와 OFF 20회, 총 40회의 시험에서 모두 성공함. 이를 통해 최종 신뢰성 향상은 Servo Torque 증가뿐 아니라 Actuator Capability, Mechanical Mounting 및 Control Angle Calibration이 함께 작용한 결과로 판단함.

기능 구현 이후에는 동일한 Raspberry Pi-ESP32 Local Wi-Fi 환경에서 HTTP, MQTT QoS 0, MQTT QoS 1의 Application RTT를 비교함. Protocol 자체의 통신 특성을 실제 물리 동작시간과 분리하기 위해 Servo, LDR, Ping 및 의도적인 Control Delay가 포함되지 않는 Benchmark Path를 별도로 구성함. 각 Protocol은 50회의 Warm-up 이후 200회씩 5개의 Run을 수행하여 1000개의 Sample을 수집하였으며, 총 3000회의 Main Request가 모두 성공함.

최종 Mean Application RTT는 HTTP 20.515 ms, MQTT QoS 0 15.319 ms, MQTT QoS 1 61.282 ms였음. Median은 각각 19.450 ms, 13.988 ms, 59.508 ms였으며 P95는 28.807 ms, 22.980 ms, 79.849 ms, P99는 37.038 ms, 31.076 ms, 94.444 ms로 측정됨. 본 프로젝트의 구현 및 Local Wi-Fi 조건에서는 MQTT QoS 0가 가장 낮은 Application RTT를 보였으나, 이를 모든 환경에 적용되는 일반적인 Protocol 성능 순위로 해석하지 않음.

MQTT 초기 시험에서는 QoS 0에서도 예상보다 높은 약 100 ms 수준의 RTT가 반복적으로 관찰됨. 처음에는 Broker 또는 MQTT 처리 자체가 원인일 가능성을 고려하였으나, ESP32 Wi-Fi Power Saving 설정이 Latency에 영향을 줄 수 있다고 판단하여 별도의 A/B Test를 수행함. Wi-Fi Sleep ON 조건의 MQTT QoS 0 평균 RTT는 121.279 ms였으며, WiFi.setSleep(false)를 적용한 Sleep OFF

조건에서는 15.886 ms로 감소함. 이후 최종 MQTT QoS 0 1000회 Main Benchmark에서도 평균 15.319 ms가 측정되어 유사한 Latency 수준이 다시 관찰됨.

전체 Application RTT와 별도로 ESP32 Benchmark Handler 내부 Processing Time을 측정한 결과 Mean은 HTTP 401.999 μ s, MQTT QoS 0 419.632 μ s, MQTT QoS 1 420.765 μ s였음. 세 Protocol 모두 약 0.4 ms 수준으로 비슷하였기 때문에 Protocol에 따라 나타난 수십 ms 수준의 RTT 차이가 본 실험에서 측정한 ESP32 Handler 내부 계산시간에 의해 지배된 것은 아니라는 점을 확인함.

ESP32의 반복 요청에 따른 Runtime Memory 상태를 확인하기 위해 각 Request에서 Free Heap, Minimum Free Heap, Maximum Allocatable Heap을 기록함. Protocol별 Data의 시작과 종료값만 비교하는 방식이 실제 Interleaved Run 순서를 반영하지 못한다는 점을 확인한 후, 최종 분석에서는 총 3000개 Sample을 Timestamp 기준으로 다시 정렬함. 그 결과 Free Heap은 여러 Runtime Level 사이에서 변화하였으나 실험 전체에서 지속적으로 하락하는 추세는 관찰되지 않음. 따라서 본 Benchmark 범위에서는 반복 요청에 따른 누적적인 Free Heap 감소 현상이 확인되지 않았다고 제한적으로 해석함.

Raspberry Pi Resource Benchmark에서는 Python Worker와 Mosquitto의 CPU 및 RSS를 측정함. 현재의 0.2초 간격 Sequential Workload에서 Python Worker의 평균 CPU는 약 0.4%, RSS는 약 23 MiB 수준으로 세 Protocol 간 차이가 작았음. Mosquitto의 CPU Activity는 MQTT 조건에서 증가하였지만 절대적인 사용량은 낮았으며 RSS는 약 8.406 MiB 수준으로 유지됨.

실제 Light ON/OFF Endpoint를 대상으로 End-to-End Application Completion Time을 추가로 측정한 결과 전체 Mean은 1032.247 ms였음. Firmware에서 의도적으로 설정한 Press Hold 400 ms와 Return Wait 600 ms의 합인 1000 ms가 전체 Mean의 약 96.9%를 차지함. 따라서 실제 Light Control에서 사용자가 체감하는 약 1초의 응답시간은 수십 ms 수준의 Communication RTT보다 Programmed Actuator Sequence의 영향을 훨씬 크게 받는 것으로 나타남.

본 프로젝트에서는 결과가 예상과 다르게 나타났을 때 단순히 결과값을 받아들이는 대신 실험 조건과 측정 방법을 다시 확인함. Wi-Fi Sleep 조건을 분리하여 재실험하였고, 실제 Persistent Connection이 성립하지 않은 HTTP Keep-Alive Data는 최종 비교에서 제외하였으며, Protocol별 RSS 비교 조건이 적절하지 않았던 첫 번째 Resource Experiment도 수정 후 다시 측정함. 또한 Heap Data의 분석 순서가 실제 Experiment Order와 맞지 않는 문제를 발견한 뒤 Raw Data를 기준으로 분석 방법을 변경함.

이를 통해 본 프로젝트에서는 실제 Hardware System을 구현하는 경험뿐 아니라 측정 대상과 범위를 정의하고, 예상과 다른 결과의 원인을 추적하며, 실험 조건을 검증하고, 확인한 범위 내에서 결과를 해석하는 기초적인 시스템 성능 분석 과정을 경험함.

1. 프로젝트 개요

1.1 프로젝트 배경

1.1.1 초기 프로젝트 동기

본 프로젝트는 Raspberry Pi와 ESP32를 직접 사용하여 실제 생활 환경의 장치를 원격으로 제어해보는 것을 초기 목표로 시작함. 단순한 LED 점등이나 가상의 **Software State** 변경보다 실제로 생활에서 사용할 수 있는 물리 장치를 제어해보고 싶었으며, 첫 번째 대상으로 방의 벽면 전등 스위치와 PC 전원 버튼을 선정함. 기존 장치의 내부 배선이나 전기 회로를 직접 변경하는 대신 **Servo Motor**를 이용하여 사람이 버튼을 누르는 동작 자체를 물리적으로 재현하는 방식을 선택함.

초기 프로젝트를 통해 다음 요소를 직접 경험하고자 함.

- Raspberry Pi 기반 Linux 환경
- ESP32 기반 Embedded Programming
- Wi-Fi Network Communication
- HTTP Client-Server Communication
- GPIO 및 Sensor/Actuator Control
- 외부 전원 및 Common GND 구성
- Sensor를 이용한 실제 상태 확인
- 실제 Hardware Control 과정에서 발생하는 문제 해결

초기에는 실제 장치가 원격 명령에 따라 정상적으로 움직이는 것 자체가 프로젝트의 가장 중요한 목표였음.

1.1.2 엣지 제어 시스템과 전체 제어 경로

본 프로젝트에서 Raspberry Pi 5는 사용자와 실제 Hardware Node 사이의 중앙 Edge Control Server로 사용함. 전체 경로는 다음과 같이 구성됨.

Client / Dashboard

→ Raspberry Pi 5

→ Local Wi-Fi Network

→ ESP32 Hardware Control Node

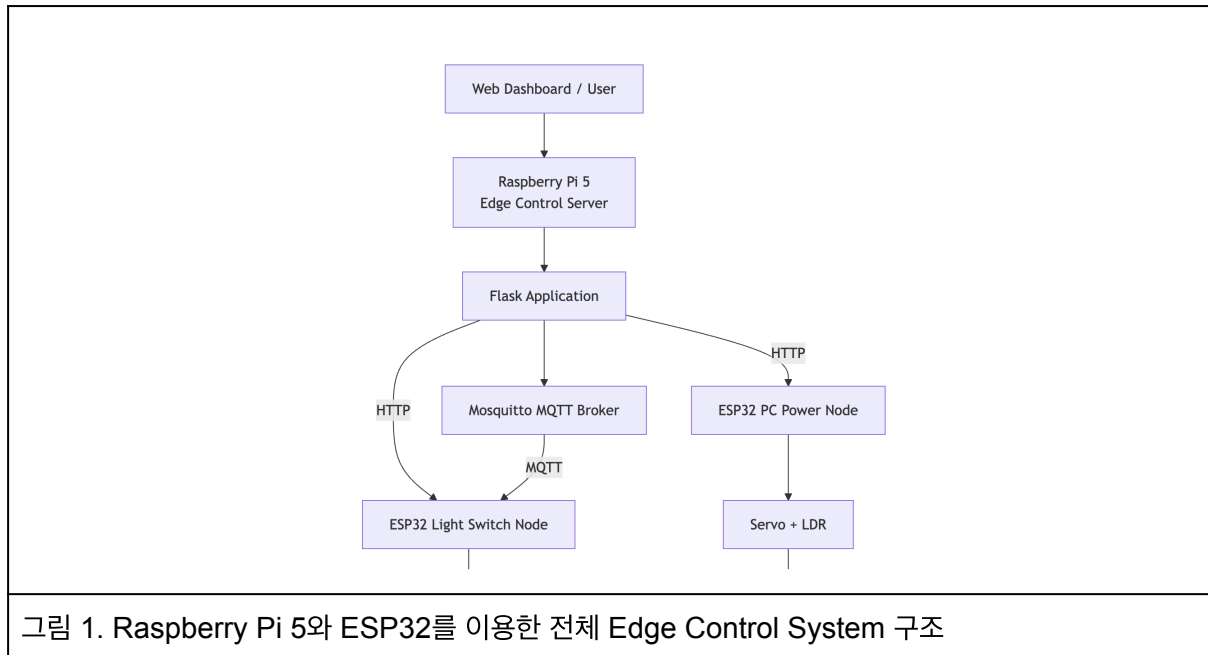
→ Sensor / Servo Motor

→ Physical Device

사용자가 Dashboard에서 명령을 입력하면 Raspberry Pi가 해당 Request를 수신함. Light Switch Control에서는 명령을 대상 ESP32에 전달하고, PC Power Control에서는 먼저 상태 확인을 수행한 뒤

필요한 경우에만 ESP32에 Physical Control을 요청함. ESP32는 실제 Hardware와 직접 연결되어 Servo를 움직이거나 LDR Sensor 값을 측정함.

초기에는 이 전체 과정을 하나의 기능으로 보았지만 프로젝트를 진행하면서 하나의 사용자 Request 안에도 Network Communication, Software Processing, Sensor Sampling, Actuator Control과 같이 성격이 서로 다른 단계가 존재한다는 점에 관심을 가지게 됨.



1.1.3 기능 구현에서 성능 측정 및 분석으로의 확장

Light Switch와 PC Power 기능을 구현한 이후에는 단순히 기능을 더 추가하는 것보다 현재 시스템이 실제로 어떻게 동작하는지를 확인해보고 싶어짐. 특히 Light Switch에서는 Network Request와 Servo Command가 정상임에도 실제 Switch가 눌리지 않는 문제가 발생함. 이 과정에서 Software의 정상 동작과 실제 Physical Control Success가 서로 다른 문제라는 점을 경험함.

기능 구현 이후에는 통신 성능에 대해서도 궁금증이 생김. Raspberry Pi가 ESP32로 명령을 보낸 뒤 Response를 받기까지 실제로 어느 정도의 시간이 필요한지, HTTP와 MQTT를 동일한 장치와 Network에서 사용하면 어떤 차이가 발생하는지 직접 확인하고자 함. 처음에는 단순한 Latency 비교로 시작하였지만 측정을 진행하면서 다음과 같은 질문으로 자연스럽게 확대됨.

- 평균 Latency만이 아니라 Tail Latency는 어떻게 나타나는가?
- MQTT QoS에 따라 Application RTT가 달라지는가?
- 전체 RTT의 차이가 ESP32 내부 처리시간에서 발생하는가?
- 반복 요청 동안 ESP32의 Heap 상태는 어떻게 변화하는가?
- Raspberry Pi와 Mosquitto가 사용하는 CPU와 Memory는 어느 정도인가?
- 실제 사용자가 체감하는 전체 Control Time은 통신시간과 같은가?

이후 프로젝트 후반부에서는 새로운 기능을 계속 늘리기보다 이미 구현한 시스템을 실험 대상으로 활용하여 위 질문을 확인하는 방향으로 진행함.

1.2 문제 정의

Network 기반 Physical Control System에서는 Request가 정상적으로 전달되었다는 사실만으로 전체 Control이 성공했다고 판단할 수 없음. 예를 들어 Raspberry Pi가 ESP32에 정상적으로 HTTP Request를 전달하고 ESP32가 정상적인 Response를 반환하더라도 Servo의 Torque가 부족하거나 장착 구조가 흔들리거나 Press Angle이 적절하지 않으면 실제 Switch가 동작하지 않을 수 있음.

PC Power Control에서는 반대 문제도 발생함. PC가 이미 ON 상태인데 현재 상태를 확인하지 않고 Power Button을 다시 누르면 사용자가 원하지 않는 종료 또는 절전 동작이 발생할 가능성이 있음.

Latency 역시 전체 시간을 하나의 값으로만 보면 실제 원인을 구분하기 어려움. 실제 Light Control이 약 1초 걸린다고 해서 Network가 1초 동안 동작한다고 볼 수는 없음. Servo의 Press Hold와 Return Wait가 Firmware에 의도적으로 설정되어 있다면 전체 사용자 체감시간 중 상당 부분은 해당 Control Policy 때문일 수 있음.

이에 본 프로젝트에서는 문제를 크게 다음과 같이 구분함.

1. Communication

- Raspberry Pi와 ESP32 사이의 HTTP/MQTT 통신
- Wi-Fi 설정
- Application RTT

2. Processing 및 Runtime Resource

- ESP32 Handler Processing
- ESP32 Heap
- Raspberry Pi CPU/RSS
- Mosquitto CPU/RSS

3. 상태 판단

- PC Ping
- LDR Sampling
- Ping과 LDR을 결합한 조건부 Control

4. Physical Actuation

- Servo Motor
- Torque
- Mounting
- Press Angle
- 실제 Switch 동작

이와 같은 구분을 통해 특정 실패나 지연이 발생했을 때 전체 시스템을 단순히 느리거나 불안정하다고 판단하는 대신 어느 부분에서 문제가 나타나는지를 확인하고자 함.

1.3 프로젝트 목표

1.3.1 실제 하드웨어 제어 시스템 구현

Raspberry Pi 5를 중앙 Edge Control Server로 구성하고 ESP32를 Wireless Hardware Control Node로 사용함. Light Switch Node에서는 실제 벽면 전등 스위치를 Servo Motor로 제어함. PC Power Node에서는 PC의 현재 상태를 확인한 뒤 필요한 경우에만 Servo를 이용하여 실제 Power Button을 누르도록 구성함.

1.3.2 상태 검증 기반 조건부 제어

PC Power Node에서는 Power Request가 들어왔다는 이유만으로 무조건 Servo를 동작시키지 않음. Raspberry Pi의 Ping 결과와 ESP32의 LDR Sensor 결과를 함께 사용함. 판단 기준은 다음과 같음.

Ping 성공 OR LDR LED ON	→ PC ON
Ping 실패 AND LDR LED OFF	→ OFF_CANDIDATE
OFF_CANDIDATE	→ Physical Power Control 허용

이를 통해 Network 기반 상태 정보와 실제 Power LED의 Physical State를 함께 활용함.

1.3.3 통신 방식에 따른 성능 비교

초기 시스템에서는 Raspberry Pi와 ESP32 사이의 통신에 HTTP를 이용함. 후속 실험에서는 MQTT를 추가하여 다음 세 조건을 비교함.

- HTTP
- MQTT QoS 0
- MQTT QoS 1

단순 Mean뿐 아니라 다음 값을 함께 분석함.

- Median
- Standard Deviation
- Min / Max
- P95
- P99
- Success Rate
- Run별 결과

1.3.4 제어 경로의 지연 구성 분석

전체 Application RTT 하나만으로는 어느 부분에서 시간이 사용되는지 알기 어려움. 따라서 가능한 범위에서 ESP32 내부 Handler Processing Time을 별도로 측정하고, 실제 Light Control에서는 Firmware의 Programmed Actuator Delay와 End-to-End Time을 비교함.

다만 본 프로젝트에서는 전체 경로의 모든 Component를 각각 독립적인 Latency 값으로 실측하지는 않음. 이후 3.7절에서 전체 경로를 개념적으로 분해하고, 실제로 측정한 범위를 따로 구분함.

1.3.5 임베디드 및 엣지 장치의 자원 상태 관찰

ESP32에서는 다음 값을 기록함.

- Free Heap
- Minimum Free Heap
- Maximum Allocatable Heap
- RSSI

Raspberry Pi에서는 다음 Process Resource를 확인함.

- Python Worker CPU
- Python Worker RSS
- Mosquitto CPU
- Mosquitto RSS

본 측정은 CPU Cache, DRAM, Memory Controller와 같은 좁은 의미의 Memory System 분석을 수행한 것은 아니며, 현재 Application과 반복 Workload에서 나타나는 기초적인 Runtime Resource Behavior를 확인하는 범위로 한정함.

1.4 분석 질문

본 프로젝트의 후속 실험은 다음 질문을 중심으로 진행함.

Q1. 동일한 Raspberry Pi-ESP32 Local Wi-Fi 환경에서 HTTP, MQTT QoS 0, MQTT QoS 1의 Application RTT는 어떻게 다른가?

Q2. Protocol별 Application RTT 차이는 ESP32 Benchmark Handler 내부 Processing Time으로 설명할 수 있는가?

Q3. 초기 MQTT 시험에서 예상보다 높게 나타난 Latency에 ESP32 Wi-Fi Sleep 설정이 영향을 주는가?

Q4. 전체 Main Benchmark 동안 ESP32 Free Heap은 지속적인 감소 추세를 보이는가?

Q5. 현재 Sequential Workload에서 HTTP와 MQTT 사용 시 Raspberry Pi의 CPU 및 RSS는 어느 정도인가?

Q6. 실제 Light Switch Control에서 사용자가 체감하는 End-to-End Time의 대부분은 어떤 요소가 차지하는가?

Q7. 초기 Light Switch의 낮은 Physical Success Rate는 어떤 방식으로 개선할 수 있는가?

1.5 프로젝트 범위

최종 구현 범위는 다음과 같음.

- Raspberry Pi 5 Edge Control Server
- Flask Dashboard
- ESP32 Light Switch Node
- ESP32 PC Power Node
- HTTP 기반 Control
- HTTP Benchmark
- MQTT QoS 0 / QoS 1 Benchmark
- MQTT Application ACK
- Servo Motor Control
- LDR Sensor
- Ping 기반 상태 확인
- Ping + LDR 기반 조건부 PC Control
- Local Button Control

최종 측정 범위는 다음과 같음.

- HTTP Application RTT
- MQTT QoS 0 Application RTT
- MQTT QoS 1 Application RTT
- Latency Distribution 및 Tail Latency
- ESP32 Benchmark Handler Processing Time
- ESP32 Heap 변화
- Raspberry Pi Process Resource Usage
- Wi-Fi Sleep ON/OFF Diagnostic
- Light Switch End-to-End Application Completion Time
- Physical Actuation Success Rate
- Servo Short-Term Repetition Stability

1.6 본 프로젝트에서 직접 측정하지 않은 범위

측정한 범위와 추정한 범위를 혼동하지 않기 위해 다음 항목은 본 프로젝트에서 직접 독립적으로 측정하지 않았음을 명시함.

- Browser Client에서 Raspberry Pi까지의 독립 Latency
- 순수 Wi-Fi Propagation Latency
- TCP 내부 Stage별 Latency
- MQTT Broker 내부 Stage별 처리시간
- QoS 1의 PUBACK 자체만의 독립 RTT
- PC Power Node 전체 End-to-End Time
- 실제 Switch Contact 순간을 External Sensor로 Timestamp한 Physical Latency
- Packet Loss 환경에서의 QoS Reliability 차이
- Maximum Throughput 또는 Concurrent Request 성능

따라서 이후의 결과 해석에서도 이러한 값을 실제로 측정한 것처럼 표현하지 않음.

2. 시스템 구성

2.1 전체 하드웨어 구성

주요 Hardware는 다음과 같음.

부품	역할
Raspberry Pi 5 8GB	중앙 Edge Control Server
ESP32-DEVKITC-32E	Wireless Hardware Control Node
SG90 Servo Motor	PC Power Button Control
MG90S Servo Motor	초기 Light Switch Control
MG996R Servo Motor	최종 Light Switch Control
LDR Sensor Module	PC Power LED State Measurement
5V 5A External Power Adapter	Servo Power
Tact Switch	Local Control Input
Breadboard / Jumper Wires	Circuit 구성
Raspberry Pi Cooling System	Raspberry Pi 발열 관리



그림 2. Raspberry Pi 5, ESP32 및 실제 제어 장치로 구성된 전체 Hardware

2.2 Raspberry Pi 5 엣지 제어 서버

Raspberry Pi 5는 전체 시스템의 중앙 Control Point로 사용함. 주요 역할은 다음과 같음.

- Flask Server 실행
- Dashboard 제공
- 사용자 Request 수신
- ESP32로 HTTP Command 전달
- PC Ping 수행
- ESP32 LDR State 조회
- 조건부 PC Control 판단
- Mosquitto Broker 실행
- Protocol Benchmark Script 실행
- Resource Measurement Script 실행
- Raw Data 저장 및 분석

Raspberry Pi에 모든 Sensor와 Actuator를 직접 연결하지 않고 실제 Hardware 주변에 ESP32를 배치하여 역할을 분리함.

2.3 ESP32 하드웨어 제어 노드

ESP32는 실제 Hardware와 직접 연결되어 Raspberry Pi로부터 전달된 Command를 Physical Operation으로 변환함. Light Switch Node의 주요 기능은 다음과 같음.

- Wi-Fi 연결
- HTTP Command 처리
- Servo Motor Control
- Local Button Input
- HTTP Benchmark
- MQTT Benchmark
- Heap 상태 기록
- RSSI 기록
- ESP32 내부 Processing Time 기록

PC Power Node에서는 다음 기능을 구현함.

- Wi-Fi 연결
- LDR ADC Sampling
- PC Power LED 상태 판단
- Servo Power Button Control
- Command Result 반환

2.4 서보모터 및 외부 전원 구성

Servo Motor는 동작 순간 비교적 높은 전류를 요구할 수 있으므로 ESP32의 전원 Pin만으로 구동하지 않고 별도의 5V External Power를 사용함. ESP32의 Control Signal과 Servo가 동일한 기준 전위를 사용하도록 외부 전원 GND와 ESP32 GND를 Common Ground로 연결함.

Light Switch의 초기 Prototype에서는 MG90S를 사용하였으나 실제 벽면 Switch를 안정적으로 누르지 못하는 문제가 발생함. 이후 최종 구성에서는 MG996R을 사용하고 Mounting 및 Angle을 함께 수정함.

2.5 PC 전원 상태 검증을 위한 LDR 및 Ping

PC 상태를 하나의 Signal만으로 판단하지 않도록 Network State와 Physical State를 함께 이용함. Ping은 Raspberry Pi에서 수행함. LDR는 ESP32가 PC Power LED의 밝기를 Analog Value로 측정함.

현재 사용한 LDR 구성에서는 빛이 강할수록 Raw Value가 작게 나타남. 실제 관찰 범위는 다음과 같음.

PC ON	→ 약 57 ~ 2905
PC OFF	→ 약 4095
Threshold	→ 3500

LDR는 20개의 Sample을 사용하며 Sample 사이에 5 ms Delay를 설정함. 따라서 Firmware상 의도적인 Sampling Delay는 약 100 ms임.

PC 상태 판단은 다음과 같음.

Ping	LDR	판단
Success	ON	PC ON
Success	OFF	PC ON
Fall	ON	PC ON
Fail	OFF	OFF_CANDIDATE

Ping Failure 하나만으로 PC OFF를 결정하지 않고, LDR까지 OFF를 나타낼 때만 Power Control을 허용함.



그림 3. PC Power LED의 Physical State를 측정하는 LDR와 Power Button Servo

2.6 소프트웨어 환경

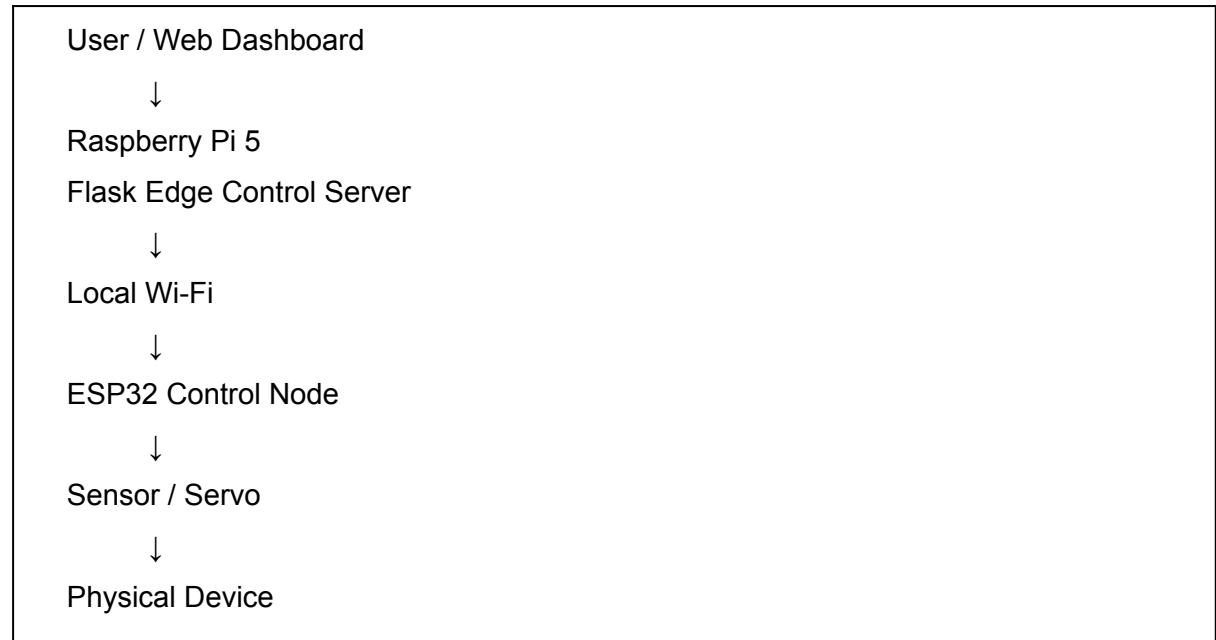
최종 실험 및 분석에서 확인한 주요 환경은 다음과 같음.

Raspberry Pi OS	→ Debian GNU/Linux 13 trixie 기반 환경
Python	→ 3.13.5
pandas	→ 3.0.5
NumPy	→ 2.5.2
Matplotlib	→ 3.11.1
psutil	→ 7.2.2
paho-mqtt	→ 2.1.0
Mosquitto Broker	→ 2.0.21
Arduino IDE	→ 2.3.10
ESP32 Board	→ ESP32 Dev Module
ESP32 MQTT Library	→ MQTT by Joel Gaehwiler 2.5.3
Serial Baud Rate	→ 115200

3. 시스템 설계

3.1 전체 시스템 구조

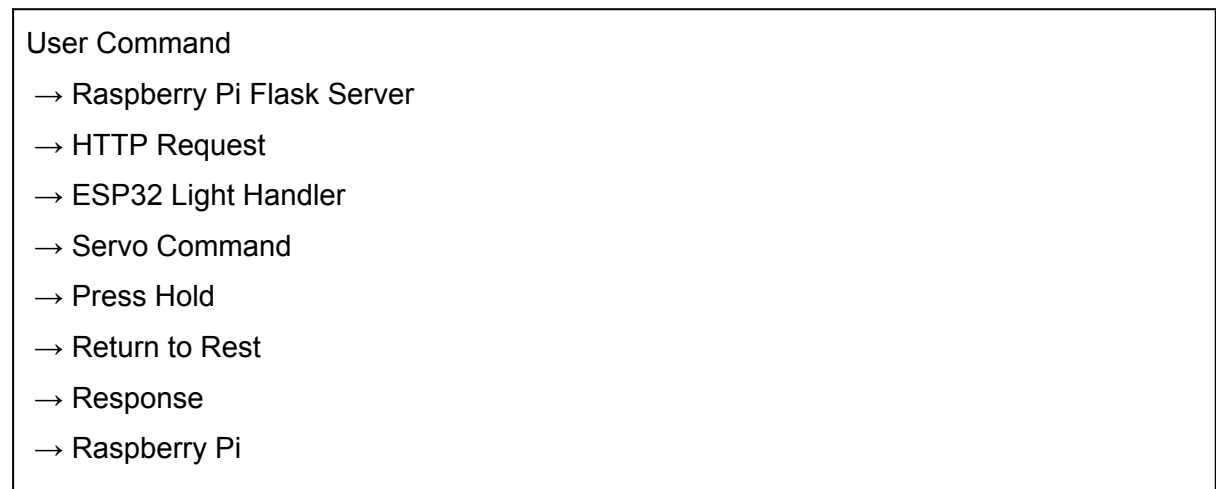
전체 시스템은 다음과 같은 형태로 구성함.



Light Switch와 PC Power Node는 동일한 Raspberry Pi를 중앙 Server로 사용하지만 실제 Control Logic은 서로 다름. Light Switch는 비교적 단순한 Command → Servo Control 구조이며, PC Power는 상태 확인이 선행되는 조건부 제어 구조임.

3.2 Light Switch Node 제어 경로

실제 Light Switch의 기본 Control Path는 다음과 같음.



최종 Light Servo Parameter는 다음과 같음.

REST Angle	→ 90°
ON Angle	→ 50°
OFF Angle	→ 140°
Press Hold	→ 400 ms
Return Wait	→ 600 ms
Servo Pulse Range	→ 500 ~ 2400 μ s

실제 Control Path에는 Physical Actuation을 위해 약 1000 ms의 Programmed Delay가 포함됨.

3.3 PC Power Node 제어 경로

PC Power Control은 상태 확인이 먼저 수행되기 때문에 Light Switch보다 복잡함.

User Power Request

- Raspberry Pi
- Ping PC
- ESP32 LDR State Request
- Ping + LDR Combined Decision
- PC ON 여부 판단
- OFF_CANDIDATE인 경우 Servo Press
- Return
- 이후 상태 확인

PC Servo Control에는 코드상 약 1950 ms의 Programmed Actuator Sequence가 포함됨. 이 값은 Firmware에 설정된 Sequence를 기준으로 한 것이며 실제 PC End-to-End Latency를 측정한 결과는 아님.

3.4 상태 검증 기반 조건부 제어

PC Power Control에서 가장 중요한 설계 기준은 “Command가 들어왔다”와 “Command를 실제로 수행해야 한다”를 구분하는 것이었음.

Ping이 성공하면 PC가 Network에 응답 가능한 상태로 판단함. LDR에서 Power LED가 ON으로 관찰될 경우 Network 상태와 관계없이 PC가 켜져 있는 것으로 판단함. 따라서 둘 중 하나라도 ON을 나타내면 Physical Power Control을 차단함. 두 Signal이 모두 OFF를 나타낼 때만 OFF_CANDIDATE로 판단하여 Servo Control을 허용함.

이 방식이 PC 상태를 완벽하게 판별한다고 주장할 수는 없지만, 하나의 Signal만으로 판단하는 것보다 불필요한 Power Button 동작을 줄이기 위한 구조로 구현함.

3.5 HTTP 기반 통신 구조

초기 Raspberry Pi-ESP32 통신에서는 ESP32가 HTTP Server로 동작함. Raspberry Pi는 특정 Endpoint로 HTTP Request를 전달하고 ESP32는 JSON Response를 반환함. HTTP Application RTT의 측정 범위는 다음과 같음.

Raspberry Pi HTTP Request Start

- Host HTTP Processing
- Wi-Fi / TCP
- ESP32 HTTP Server 및 Handler
- JSON Response
- Wi-Fi / TCP Return
- Raspberry Pi Response Body Read Complete

따라서 해당 RTT는 순수 Wi-Fi 전송시간이 아니라 HTTP Application Request/Response 전체 경로의 시간임.

3.6 MQTT 기반 Benchmark 구조

후속 실험에서는 Raspberry Pi에서 Mosquitto Broker를 실행하고 ESP32가 MQTT Benchmark Topic을 Subscribe하도록 구성함. Benchmark Topic은 다음과 같음.

Command	→ edge/light/benchmark/cmd
ACK	→ edge/light/benchmark/ack

Command Payload에는 command_id와 QoS 정보를 포함함. Raspberry Pi는 Command를 Publish하기 직전 Timer를 시작함. ESP32는 해당 Command를 처리한 뒤 동일한 command_id가 포함된 Application ACK를 Publish함. Raspberry Pi에서 해당 ACK를 수신하면 Timer를 종료함.

따라서 MQTT Application RTT는 다음 구간을 의미함.

Raspberry Pi Publish
 → Mosquitto Broker
 → ESP32
 → ESP32 Application Handler
 → Application ACK
 → Mosquitto Broker
 → Raspberry Pi Receive

QoS 1 조건에서도 측정값은 MQTT Protocol의 PUBACK 자체만의 시간이 아니라 ESP32의 Application ACK가 돌아오기까지의 전체 Application RTT임.

3.7 제어 경로 지연시간 분해 모델

사용자가 하나의 Control Command를 수행할 때 전체 시간은 여러 성격의 구간으로 구성될 수 있음. 이를 개념적으로 다음과 같이 구분함.

T_client	: Client가 요청을 전송한 뒤 Raspberry Pi가 해당 요청을 수신하기까지의 시간
T_pi	: Raspberry Pi 내부에서 Request를 해석하고 상태 확인 또는 Control Policy를 판단하는 시간
T_comm	: Raspberry Pi와 ESP32 사이의 Communication 및 Protocol 처리 구간
T_esp	: ESP32가 명령을 수신한 뒤 Handler 내부에서 수행하는 Processing Time
T_sensor	: LDR 등의 Sensor Sampling 및 상태 계산시간
T_control_policy	: Servo Hold, Return Wait, PC Power Sequence와 같이 Firmware에서 의도적으로 설정한 Control Time
T_verify	: Control 이후 실제 상태를 다시 확인하는 데 사용되는 시간

따라서 사용자 관점의 전체 시간은 개념적으로 다음과 같이 나타낼 수 있음.

$$T_{\text{user}} \approx T_{\text{client}} + T_{\text{pi}} + T_{\text{comm}} + T_{\text{esp}} + T_{\text{sensor}} + T_{\text{control_policy}} + T_{\text{verify}}$$

모든 Command가 항상 모든 Component를 포함하는 것은 아님. 예를 들어 Light Switch Control에는 PC Ping 및 LDR Sampling이 포함되지 않으며, Protocol Benchmark에서는 Servo, Sensor, Verification과 같은 Physical Control 관련 Component를 의도적으로 제외함.

3.8 실제 측정 범위와 개념적 모델의 구분

3.7절의 식은 전체 시스템을 이해하기 위한 Conceptual Decomposition Model이며 본 프로젝트에서 각각의 항목을 모두 독립적으로 측정한 것은 아님.

실제로 직접 측정한 주요 Timing은 다음과 같음.

Application RTT	→ Raspberry Pi 내부 Timer를 이용하여 HTTP Request/Response 또는 MQTT Command/Application ACK 전체 시간을 측정함.
ESP32 Handler Processing	→ ESP32 내부 micros()를 이용하여 Benchmark Handler 내부 측정 구간을 기록함.
Light End-to-End Application Completion Time	→ 실제 Light ON/OFF Endpoint의 Request 시작부터 Response 완료까지 Raspberry Pi에서 측정함.
Programmed Actuator Delay	→ Firmware에 명시된 Press Hold 400 ms와 Return Wait 600 ms를 사용함.

반면 다음 값은 독립적인 실측값으로 분리하지 못함.

T_client	→ Protocol Benchmark Script가 Raspberry Pi에서 직접 실행되므로 별도 Browser Client 구간을 측정하지 않음.
T_pi	→ 독립적인 Timing Segment로 측정하지 않음.
T_comm	→ 순수 Network Segment만 분리하지 않음.
T_sensor	→ LDR의 Firmware상 20 × 5 ms Sampling Delay는 확인 가능하지만 이를 독립적인 Sensor Processing 실측값으로 사용하지 않음.
T_verify	→ PC Control에서 상태 재확인 Logic은 존재하지만 해당 구간을 별도 Timing Result로 수집하지 않음.

따라서 본 보고서에서는 측정하지 않은 Component를 추정하여 순수 Network Latency 또는 독립 Processing Time으로 제시하지 않음.

3.9 의도된 제어 지연과 시스템 오버헤드의 구분

Light Switch Firmware에는 실제 Switch를 안정적으로 누르기 위해 다음 Delay가 의도적으로 포함되어 있음.

Press Hold	→ 400 ms
Return Wait	→ 600 ms

따라서 Light Control Function이 약 1초 이상 걸린다고 해서 이를 Network 또는 ESP32 CPU 성능의 1초 Bottleneck으로 해석할 수 없음. PC Power Node에서도 LDR Sampling Delay와 Servo Sequence 등이 존재함.

이에 Protocol Benchmark에서는 Servo, Sensor 및 Physical Control을 제외한 별도 Benchmark Path를 사용하고, 실제 사용자 체감시간을 확인할 때는 실제 Light Endpoint를 별도로 측정함. 이를 통해 Communication Performance와 User-visible Control Time을 구분함.

3.10 메모리 및 자원 분석 기준

ESP32에서는 다음 세 Memory Value를 중심으로 확인함.

Free Heap	→ 현재 사용 가능한 Heap의 총량
Minimum Free Heap	→ Boot 이후 관찰된 Free Heap의 최소값
Maximum Allocatable Heap	→ 현재 한 번에 할당할 수 있는 가장 큰 Heap Block의 크기

Free Heap 값 하나가 감소했다는 사실만으로 Memory Leak을 판단하지 않음. 또한 Minimum Free Heap은 Boot 이후의 Historical Minimum이므로 특정 Protocol의 순간 Memory Usage와 직접적으로 대응하는 값이 아님.

최종 Main Benchmark는 Protocol 순서를 Run별로 교차했기 때문에 Protocol별 CSV의 First/Last Heap을 단순 비교하지 않고 전체 3000 Sample을 실제 Timestamp 순서로 정렬하여 Runtime Trend를 확인함.

Raspberry Pi에서는 Python Worker와 Mosquitto Process의 CPU 및 RSS를 확인함. CPU 값은 0.2초의 Request Interval을 포함한 Workload Average이므로 Request 한 번의 순수 CPU Cost로 해석하지 않음.

3.11 실험 데이터 관리 구조

실험 과정에서 정상적인 Main Data뿐 아니라 Diagnostic, 조건 불확실 Data, 초기 Historical Data가 함께 생성됨. 따라서 다음과 같이 분류함.

data/raw/main/	→ 최종 분석에 사용한 데이터
data/raw/diagnostic/	→ Dry Run 및 원인 확인을 위한 데이터
data/raw/excluded/	→ 실험 조건이 성립하지 않았거나 확인되지 않아 최종 분석에서 제외한 데이터
data/raw/legacy/	→ 초기 개발 단계의 Historical Data
data/processed/	→ Raw CSV에서 생성한 Summary Data
data/figures/	→ 분석 Script를 이용하여 생성한 Figure

Raw Data는 최종 Source of Truth로 유지함.

4. 구현 과정

4.1 Raspberry Pi 초기 환경 구성

Raspberry Pi 5에 Raspberry Pi OS를 설치하고 Wi-Fi 및 SSH 기반 Headless Environment를 구성함. 이후 개발 PC에서 SSH를 이용하여 Server 실행, Code 수정, Benchmark 수행 및 Data 확인을 할 수 있도록 구성함. Raspberry Pi는 Flask Server, MQTT Broker, Benchmark Script 및 분석 Script를 실행하는 중앙 Device로 사용함.

초기 상태에서는 CPU Temperature와 Throttling 여부도 확인함. Raspberry Pi의 초기 CPU Temperature는 약 46°C 수준이었으며 Throttling 상태는 0x0으로 확인됨. 이를 통해 해당 측정 시점에서 발열 또는 저전압에 의한 Throttling이 나타나고 있지 않음을 확인함.

4.2 Flask 기반 Dashboard 및 중앙 제어 서버 구현

Raspberry Pi에서 Python Flask 기반 Web Server를 구현함. Dashboard에서 Light Switch 및 PC Power 관련 Control을 수행할 수 있도록 구성함.

Light Switch Request는 대상 ESP32에 Control Command를 전달함. PC Power Request에서는 Raspberry Pi가 먼저 PC Ping과 ESP32 LDR 상태를 확인하고, PC가 OFF_CANDIDATE로 판단된 경우에만 Power Button Command를 허용함.

따라서 Raspberry Pi는 Web Page 제공뿐 아니라 사용자 Request와 Hardware Node 사이의 중앙 Control Logic을 담당함.

4.3 ESP32 Wi-Fi 연결 및 HTTP Server 구현

ESP32를 Raspberry Pi와 동일한 Local Wi-Fi Network에 연결함. Wi-Fi Connection 이후 Raspberry Pi에서 ESP32 IP에 접근할 수 있는지 확인하고 ESP32 HTTP Server를 구현함. 초기 Function은 다음과 같이 구성함.

- Health Check
- Servo Control
- LDR State
- PC Power Control
- Memory Information

후속 실험에서는 여기에 Benchmark Endpoint와 MQTT Benchmark Logic을 추가함. ESP32 Response는 Raspberry Pi에서 처리하기 쉽게 JSON 형태로 구성함.

4.4 Light Switch Node 구현

Light Switch Node에서는 기존 벽면 전등 Switch의 내부 배선을 직접 수정하지 않고 Servo가 Switch를 물리적으로 누르도록 구현함. Remote Command와 Local Button을 모두 지원하며 두 Input은 동일한 Light Control Function을 이용하도록 구성함.

Servo가 목표 Angle로 이동하여 Switch를 누른 뒤 일정 시간 후 Rest Position으로 복귀하도록 설정함. 최종 Parameter는 다음과 같음.

REST	→ 90°
ON	→ 50°
OFF	→ 140°
Press Hold	→ 400 ms
Return Wait	→ 600 ms

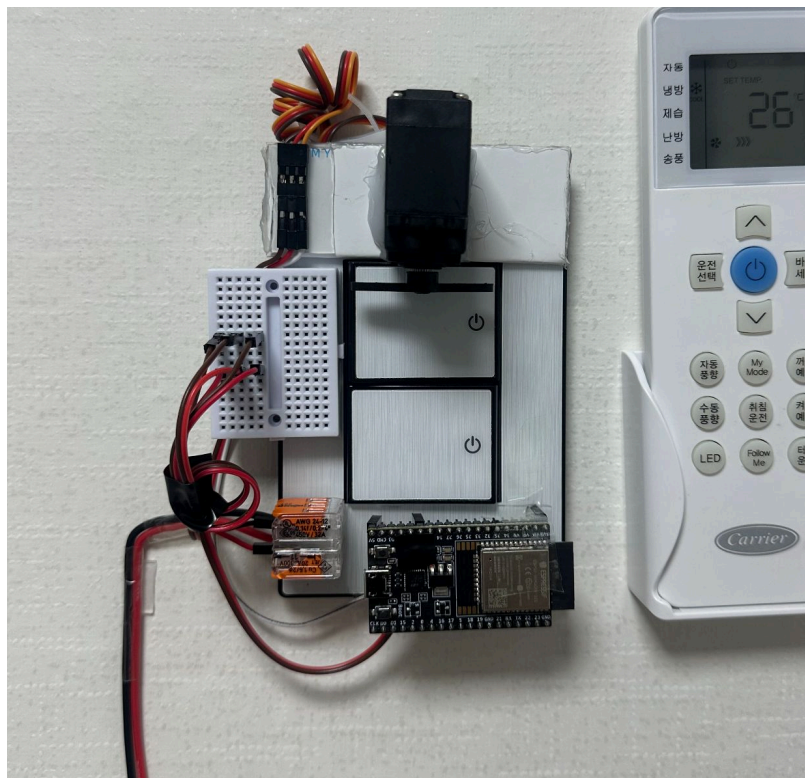


그림 4. MG996R Servo를 이용한 최종 Light Switch Control 구조

4.5 PC Power Node 구현

PC Power Node에서는 Servo를 이용하여 실제 PC Power Button을 누르도록 구성함. Light Switch와 달리 현재 PC State를 먼저 확인해야 하므로 LDR와 Ping을 추가함.

ESP32는 Power LED의 밝기를 측정하고 Raspberry Pi는 PC에 Ping을 수행함. 두 결과를 기반으로 OFF_CANDIDATE를 판단한 뒤 필요한 경우에만 Servo를 동작시킴.

4.6 Ping과 LDR을 결합한 상태 판단 구현

PC ON 상태에서 LDR Raw Value는 약 57~2905 범위가 관찰되었으며 OFF 상태에서는 약 4095 수준으로 나타남. 초기 Threshold는 3000을 사용하였으나 실제 측정 범위를 확인한 뒤 3500으로 수정함. 최종 LDR Logic은 다음과 같음.

raw < 3500	→ LED ON
raw >= 3500	→ LED OFF

Ping과 LDR 중 하나라도 ON을 나타내면 PC ON으로 판단함. 둘 모두 OFF를 나타낼 때만 OFF_CANDIDATE로 판단함. 이를 통해 Network State와 Physical LED State를 결합한 조건부 제어를 구현함.

4.7 서보모터 반복 동작 및 안정성 확인

Servo 반복 사용 시 심각한 발열 또는 ESP32 Reset이 발생하는지 간단히 확인함. 10분 동안 반복 동작을 수행하였으며 총 172 Cycle이 수행됨.

Servo 표면 온도는 약 26.5°C에서 시작하여 약 27.8°C까지 상승함. 총 변화량은 약 +1.3°C임. 실험 중 ESP32 Reset이나 Servo 동작 실패는 관찰되지 않음.

다만 본 결과는 10분 범위의 Short-Term Stability 결과이며 장기간 내구성을 검증한 것으로 해석하지 않음.

4.8 벽면 스위치 제어 실패와 물리적 문제 분석

MG90S를 이용하여 실제 벽면 Switch 제어를 반복했을 때 20회 중 7회만 성공함. Success Rate는 35.0%였음.

이때 Raspberry Pi에서 ESP32로의 Request와 ESP32 Servo Command는 정상적으로 수행되고 있었음. 즉 Network Communication이 성공하였음에도 실제 Switch가 충분히 눌리지 않는 경우가 반복됨.

처음에는 Servo Torque가 부족한 것이 가장 큰 원인이라고 생각함. 그러나 실제 장치를 반복해서 관찰하면서 Servo 자체가 충분히 고정되어 있는지, Servo Horn이 Switch의 어느 부분과 접촉하는지, Press Angle이 충분한지도 결과에 영향을 준다는 것을 확인함. 따라서 문제를 Network 또는 Software Failure가 아니라 Physical Actuation Layer의 문제로 좁혀감.

4.9 MG996R 교체 및 제어 각도 보정

초기 MG90S의 Physical Success Rate가 충분하지 않아 최종 Light Switch Servo를 MG996R로 변경함. 그러나 더 높은 Torque를 가진 Servo로 교체했다고 해서 바로 안정적인 Control이 이루어진 것은 아니었음.

초기에는 Switch를 필요 이상으로 깊게 누르는 문제가 발생함. Angle을 줄이면 반대로 Servo Arm이 Switch에 닿지 않거나 접촉은 하지만 완전히 전환되지 않는 경우가 나타남.

이에 Servo Angle을 5° 또는 10° 단위로 변경하면서 실제 접촉 상태를 반복적으로 확인함. 최종 설정은 다음과 같음.

Servo	→ MG996R
REST	→ 90°
ON	→ 50°
OFF	→ 140°
Press Hold	→ 400 ms
Return Wait	→ 600 ms

최종 설정에서 실제 Switch를 ON 20회, OFF 20회 반복함.

ON	→ 20 / 20
OFF	→ 20 / 20
Total	→ 40 / 40

최종 시험에서는 모든 Control이 성공함. 다만 이 과정에서는 Servo Model, Mounting 및 Angle이 함께 개선됨. 따라서 성공률 증가를 Torque 하나만의 효과라고 분리하여 해석하지 않고 Physical Actuation Layer 전체의 개선 결과로 판단함.

4.10 MQTT Benchmark 구조 추가

기존 Light Control 기능을 유지한 상태에서 MQTT Benchmark 기능을 추가함. Raspberry Pi에서는 Mosquitto Broker를 실행하고 paho-mqtt Client를 사용함. ESP32에서는 MQTT Library를 이용해 Benchmark Command Topic을 Subscribe함.

MQTT Callback 내부에서 바로 복잡한 처리를 수행하기보다 수신한 Command에 대응하는 ACK 정보를 준비하고 Main Loop에서 ACK를 Publish하도록 구성함. MQTT QoS 0와 QoS 1 모두 동일한 command_id 기반 Application ACK 구조를 사용함. 이를 통해 Raspberry Pi에서 전송한 개별 Command와 ESP32가 처리한 ACK를 정확하게 대응시킬 수 있도록 함.

4.11 성능 측정을 위한 Instrumentation 추가

Protocol 성능을 Physical Actuation과 분리하기 위해 별도 Benchmark Endpoint를 추가함. HTTP Benchmark에서는 Request 시작부터 Response Body Read 완료까지의 Application RTT를 Raspberry Pi에서 측정함. MQTT에서는 Publish 직전부터 동일 command_id를 가진 Application ACK 수신 시점까지 측정함.

ESP32에서는 Handler 내부에 micros() 기반 Timer를 추가하여 다음 정보를 Response 또는 ACK에 포함함.

- command_id
- status
- esp_processing_us
- free_heap
- min_free_heap
- max_alloc_heap
- rssi_dbm

ESP32의 esp_processing_us는 Benchmark Handler에서 정의한 내부 측정구간임. 이 값에는 전체 Network Round Trip이나 물리 Servo Sequence가 포함되지 않음.

4.12 실험 데이터 및 분석 구조 정리

실험이 증가하면서 Raw Data와 Diagnostic Data를 하나의 폴더에 섞어두면 최종 분석에 어떤 File을 사용했는지 혼동할 수 있다고 판단함. 이에 Main, Diagnostic, Excluded, Legacy를 분리함. 최종 Main Protocol Raw Data는 다음 세 File임.

```
http_main_1000_nosleep_20260904_102455.csv  
mqtt_qos0_main_1000_nosleep_20260904_102455.csv  
mqtt_qos1_main_1000_nosleep_20260904_102455.csv
```

최종 Raspberry Pi Resource Raw Data:

```
pi_resource_samples_20260906_002151.csv
```

최종 Light E2E Raw Data:

```
light_http_e2e_20_20260906_010354.csv
```

Processed Data와 Figure는 별도 Analysis Script에서 생성함.

5. 실험 설계 및 측정 방법

5.1 실험 설계 원칙

최종 실험에서는 다음 원칙을 적용함.

1. 동일한 Raspberry Pi와 ESP32를 사용함.
2. 동일한 Local Wi-Fi 환경에서 비교함.
3. Protocol Benchmark에서 Servo, LDR, Ping과 같은 Physical Control 요소를 제외함.
4. 모든 최종 Protocol Benchmark에서 ESP32 Wi-Fi Sleep을 OFF로 통일함.
5. 각 Request는 Sequential하게 수행함.
6. 본실험 전 Warm-up Request를 수행함.
7. 하나의 Run 결과에 의존하지 않고 여러 Run을 수행함.
8. Protocol이 항상 같은 순서로 측정되지 않도록 Run별 Order를 변경함.
9. Mean뿐 아니라 Median, Standard Deviation, P95, P99, Min, Max를 함께 확인함.
10. 정상적인 실험 조건에서 발생한 큰 Latency Sample을 임의로 제거하지 않음.
11. 실험 조건 자체가 성립하지 않거나 확인할 수 없는 Data는 최종 통계에서 제외함.
12. Raw CSV를 Source of Truth로 유지함.

5.2 기존 HTTP Baseline

프로젝트 초기 HTTP 측정에서는 Raspberry Pi에서 ESP32로 100개의 Sequential Request를 수행함. 이 Test는 최종 Benchmark보다 이전 Firmware와 Endpoint를 이용한 Historical Baseline임.

Request	→ 100
Success	→ 100
Mean	→ 114.017 ms
Median	→ 114.560 ms
Min	→ 82.420 ms
Max	→ 126.070 ms

현재 최종 HTTP Benchmark와 Network 및 Firmware 조건이 다르므로 직접적인 성능 비교에는 사용하지 않음.

5.3 Wi-Fi Sleep Diagnostic

초기 MQTT QoS 0 측정에서 약 100 ms 이상의 예상하지 못한 RTT가 반복됨. 이에 ESP32 Wi-Fi Sleep을 실험 변수로 두어 A/B Test를 수행함.

각 조건에서 30개의 Request를 확인함. Sleep ON 및 Sleep OFF의 다른 MQTT Benchmark 조건은 가능한 동일하게 유지함. Sleep OFF에는 다음 설정을 적용함.

```
WiFi.setSleep(false);
```

5.4 최종 HTTP/MQTT Protocol Benchmark

최종 비교 Protocol은 다음과 같음.

- HTTP
- MQTT QoS 0
- MQTT QoS 1

각 Protocol에서:

Warm-up	→ 50 Requests
Main	→ 200 Requests × 5 Runs
Total	→ 1000 Main Samples / Protocol
전체 Main Sample:	→ 3000
Request Interval:	→ 0.2 s
Block 사이 Pause:	→ 3 s

실행은 Sequential 방식으로 수행함. Run 순서는 다음과 같음.

Run 1	HTTP → MQTT QoS 0 → MQTT QoS 1
Run 2	MQTT QoS 0 → MQTT QoS 1 → HTTP
Run 3	MQTT QoS 1 → HTTP → MQTT QoS 0
Run 4	HTTP → MQTT QoS 1 → MQTT QoS 0
Run 5	MQTT QoS 1 → MQTT QoS 0 → HTTP

Protocol별 1000개의 command_id가 모두 Unique한지, Success Sample에서 Missing RTT가 없는지, 각 Run에 200개의 Sample이 존재하는지도 분석 Script에서 확인함.

5.5 ESP32 내부 Processing Time 측정

Protocol Benchmark Request마다 ESP32 내부 Handler Processing Time을 기록함. HTTP와 MQTT 모두 Servo 및 Sensor 동작을 포함하지 않는 Benchmark Path에서 측정함. ESP32 내부에서는 micros()를 사용함.

Raspberry Pi와 ESP32가 동일한 Clock을 공유하지 않기 때문에 ESP32 Timestamp와 Raspberry Pi Timestamp의 Absolute Difference를 이용하지 않음. 각 Device 내부에서 완결되는 Measurement만 해당 Device의 Clock으로 계산함.

5.6 ESP32 Heap 변화 측정

각 Main Benchmark Request에서 다음 정보를 기록함.

- Free Heap
- Minimum Free Heap
- Maximum Allocatable Heap
- RSSI

Main Benchmark가 Protocol별 1000회를 한 번에 실행한 구조가 아니라 Run별로 Interleaved되어 있다는 점을 고려하여 최종 분석에서는 전체 3000 Sample을 Timestamp 기준으로 정렬함.

5.7 Raspberry Pi 자원 사용량 측정

Raspberry Pi Resource Benchmark는 Main Protocol Benchmark와 별도로 수행함.

Run 순서는 다음과 같음.

Run 1	HTTP → MQTT QoS 0 → MQTT QoS 1
Run 2	MQTT QoS 0 → MQTT QoS 1 → HTTP
Run 3	MQTT QoS 1 → HTTP → MQTT QoS 0

측정 항목은 다음과 같음.

- Python Worker CPU
- Python Worker RSS
- Mosquitto CPU
- Mosquitto RSS
- System CPU

최종 비교에서는 Python Worker와 Mosquitto Process를 중심으로 해석함.

5.8 Light Switch End-to-End Latency 측정

Protocol Benchmark는 Servo를 제외하였기 때문에 실제 사용자가 체감하는 Physical Control Time과는 차이가 있음. 이를 확인하기 위해 실제 HTTP Light ON/OFF Endpoint를 대상으로 별도 E2E Test를 수행함.

ON	→ 10
OFF	→ 10
Total	→ 20

Firmware에 포함된 Programmed Servo Timing은 다음과 같음.

Press Hold	→ 400 ms
Return Wait	→ 600 ms
Total	→ 1000 ms

실제 E2E Mean과 이 값을 비교하여 Physical Control Policy가 전체 시간에서 차지하는 비중을 계산함.

CSV의 success는 HTTP API Response Success를 의미함. 이 값은 별도로 수행한 40회 Physical Actuation Reliability Test와 구분함.

5.9 액추에이터 물리 제어 성공률 측정

초기 MG90S 조건에서는 실제 Wall Switch Control을 20회 수행함. Final MG996R 조건에서는 ON 20회와 OFF 20회를 각각 수행함. 실제 Switch가 원하는 상태로 전환되었는지를 직접 확인함.

5.10 측정 지표 및 분석 방법

Mean은 전체 Sample의 평균 Latency를 의미함. Median은 Sample을 순서대로 정렬했을 때 중앙에 위치하는 값임.

P95는 약 95%의 Sample이 해당 값 이하의 Latency를 가지는 지점을 의미함. P99는 약 99%의 Sample이 해당 값 이하에 위치하는 지점임.

Standard Deviation은 Latency 값의 분산 정도를 확인하기 위해 사용함. Success Rate는 전체 Request 중 정상적으로 완료된 Request의 비율을 의미함.

Protocol 비교에서는 Mean뿐 아니라 Median, P95, P99 및 Run별 평균을 함께 확인하여 일부 Sample 또는 하나의 Run만으로 결과를 판단하지 않도록 함.

6. 기존 구현 및 초기 실험 결과

6.1 Raspberry Pi 초기 상태

Raspberry Pi의 초기 CPU Temperature는 약 46°C 수준으로 관찰됨. Throttling 상태는 0x0으로 확인됨. 따라서 해당 확인 시점에서 발열 또는 저전압으로 인한 Throttling은 관찰되지 않음.

6.2 기존 HTTP 통신 지연시간

기존 HTTP Script에서 100회의 Request를 수행함.

항목	결과
Requests	100
Success	100
Failure	0
Success Rate	100%
Mean	114.017 ms
Median	114.560 ms
Min	82.420 ms
Max	126.070 ms

해당 측정은 Request 시작부터 HTTP Response Body를 읽은 시점까지의 전체 Application RTT이므로 순수 Network Latency로 해석하지 않음. 이후 최종 실험에서는 Firmware, Benchmark Endpoint 및 Wi-Fi Condition을 통일하여 새롭게 HTTP 1000회 Main Benchmark를 수행함.

6.3 ESP32 초기 Heap 변화

프로젝트 초기 Resource 확인에서 Wi-Fi Connection 이후 약 52 KB 수준의 Free Heap 감소가 관찰됨. HTTP Server를 추가로 실행한 뒤에는 약 852 bytes 수준의 추가 감소가 기록됨. 이 초기 결과는 Network Stack 및 Server Initialization 과정에서 Memory Allocation이 발생한다는 것을 관찰한 값임.

다만 특정 두 시점의 Heap Difference만으로 Runtime Stability 또는 Memory Leak을 판단할 수는 없다고 생각함. 따라서 후속 실험에서는 반복 Request 동안의 Heap Trend를 별도로 측정함.

6.4 서보모터 반복 동작 및 온도 변화

10분 동안 Servo를 반복 동작함.

항목	결과
Duration	10 min
Cycles	172
Initial Temp	약 26.5℃
Final Temp	약 27.8℃
Temperature Change	약 +1.3℃
ESP32 Reset	관찰되지 않음
Servo Failure	관찰되지 않음

해당 범위에서는 큰 Temperature Rise나 Reset이 관찰되지 않음. 장시간 내구성을 의미하지는 않음.

6.5 MG90S 벽면 스위치 제어 결과

MG90S를 이용하여 실제 벽면 Switch Control을 20회 반복함. 7회가 성공하고 13회가 실패함.

Success Rate:
35.0%

Network Request와 Servo Command가 정상적으로 처리되고 있었음에도 실제 Switch가 충분히 놀리지 않는 경우가 발생함. 이를 통해 HTTP Response Success와 실제 Physical Control Success를 동일하게 볼 수 없다는 점을 확인함.

6.6 MG996R 교체 후 제어 결과

MG996R로 Servo를 변경하고 Mounting과 Angle을 조정함. 최종 반복 결과는 다음과 같음.

방향	시도	성공	실패	성공률
ON	20	20	0	100%
OFF	20	20	0	100%
전체	40	40	0	100%

최종 조건에서는 40회의 Control이 모두 성공함. Servo Model과 함께 Mounting 및 Angle이 변경되었기 때문에 개선 효과를 Servo Torque만의 결과로 해석하지 않음.

6.7 LDR 및 Ping 기반 PC 상태 판단 결과

PC ON 상태에서 LDR Raw Value는 약 57~2905 범위로 관찰됨. PC OFF에서는 약 4095가 관찰됨. Threshold를 3500으로 설정함.

LDR:

raw < 3500	→ LED ON
raw >= 3500	→ LED OFF

Raspberry Pi에서는 Ping과 LDR 중 하나라도 ON을 나타내면 PC ON으로 판단함. 이를 통해 단일 Signal에만 의존하지 않는 상태 판단 구조를 구현함.

7. 추가 성능 실험 결과

7.1 Wi-Fi Sleep과 MQTT Latency

MQTT QoS 0를 처음 측정했을 때 예상보다 높은 약 100 ms 수준의 RTT가 반복적으로 나타남. 단순히 MQTT가 현재 환경에서 느리다고 결론내리기 전에 ESP32의 Wi-Fi Configuration이 결과에 영향을 줄 가능성을 확인함. 특히 Wi-Fi Power Saving이 Latency에 영향을 줄 수 있다고 보고 Wi-Fi Sleep ON/OFF를 비교함.

결과는 다음과 같음.

Condition	Success	Mean	Min	Max
Wi-Fi Sleep ON	30 / 30	121.279 ms	16.126 ms	334.085 ms
Wi-Fi Sleep OFF	30 / 30	15.886 ms	8.794 ms	56.288 ms

Sleep OFF에서 Mean RTT가 크게 감소함. 이에 최종 Protocol Benchmark에서는 모든 Protocol의 Wi-Fi Sleep을 OFF로 통일함. 이후 MQTT QoS 0 1000회 Main Benchmark에서 Mean 15.319 ms가 측정되어 Sleep OFF Diagnostic에서 관찰한 약 15~16 ms 수준이 다시 나타남.

따라서 본 시스템에서는 ESP32 Wi-Fi Power Saving Configuration이 Application RTT에 큰 영향을 줄 수 있다는 것을 확인함.

7.2 HTTP 및 MQTT Application RTT 비교

최종 Main Benchmark 결과는 다음과 같음.

Protocol	HTTP	MQTT QoS 0	MQTT QoS 1
N	1000	1000	1000
Success	1000/1000	1000/1000	1000/1000
Mean	20.515 ms	15.319 ms	61.282 ms
Median	19.450 ms	13.988 ms	59.508 ms
Std	4.377 ms	4.820 ms	8.360 ms
Min	13.100 ms	8.297 ms	12.318 ms
P95	28.807 ms	22.980 ms	79.849 ms
P99	37.038 ms	31.076 ms	94.444 ms
Max	47.467 ms	62.187 ms	105.922 ms

본 시스템의 최종 조건에서는 MQTT QoS 0가 가장 낮은 Mean 및 Median을 보임. HTTP는 MQTT QoS 0보다 높은 약 20 ms 수준의 Mean RTT를 보임. MQTT QoS 1은 세 조건 중 가장 높은 Mean 및 Tail Latency를 보임.

이 결과를 Protocol의 Header 크기나 하나의 내부 요소만으로 설명하지 않음. HTTP와 MQTT는 Connection Model, Broker 사용 여부, Library 및 Handler Implementation이 서로 다르기 때문임. 또한 QoS 1의 높은 RTT가 정확히 어느 내부 Stage에서 발생한 것인지는 본 실험에서 별도로 Timestamp하지 않았으므로 특정 단계 하나를 원인으로 단정하지 않음.

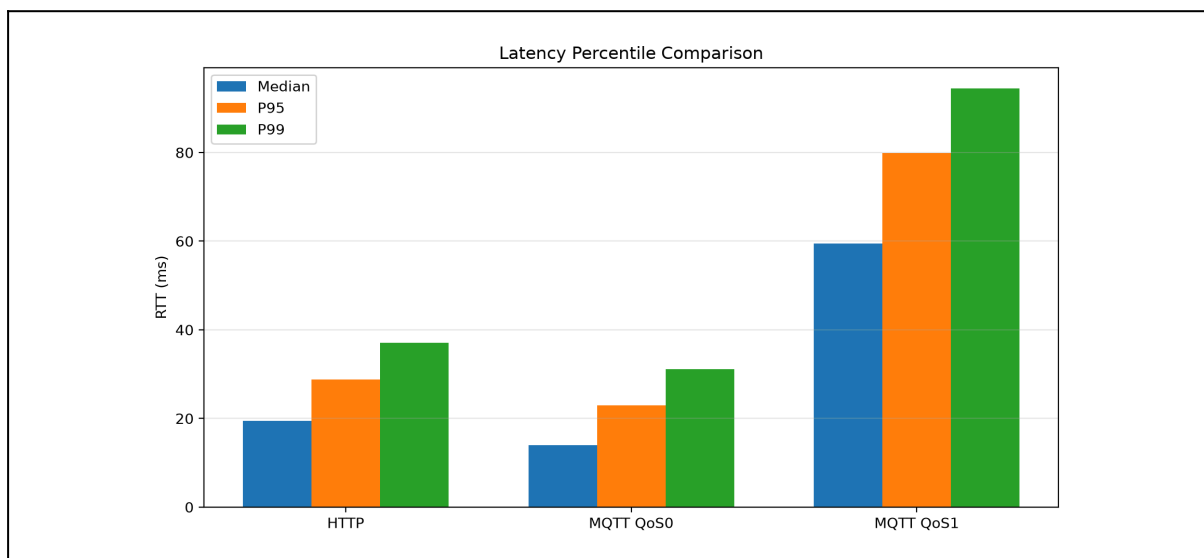


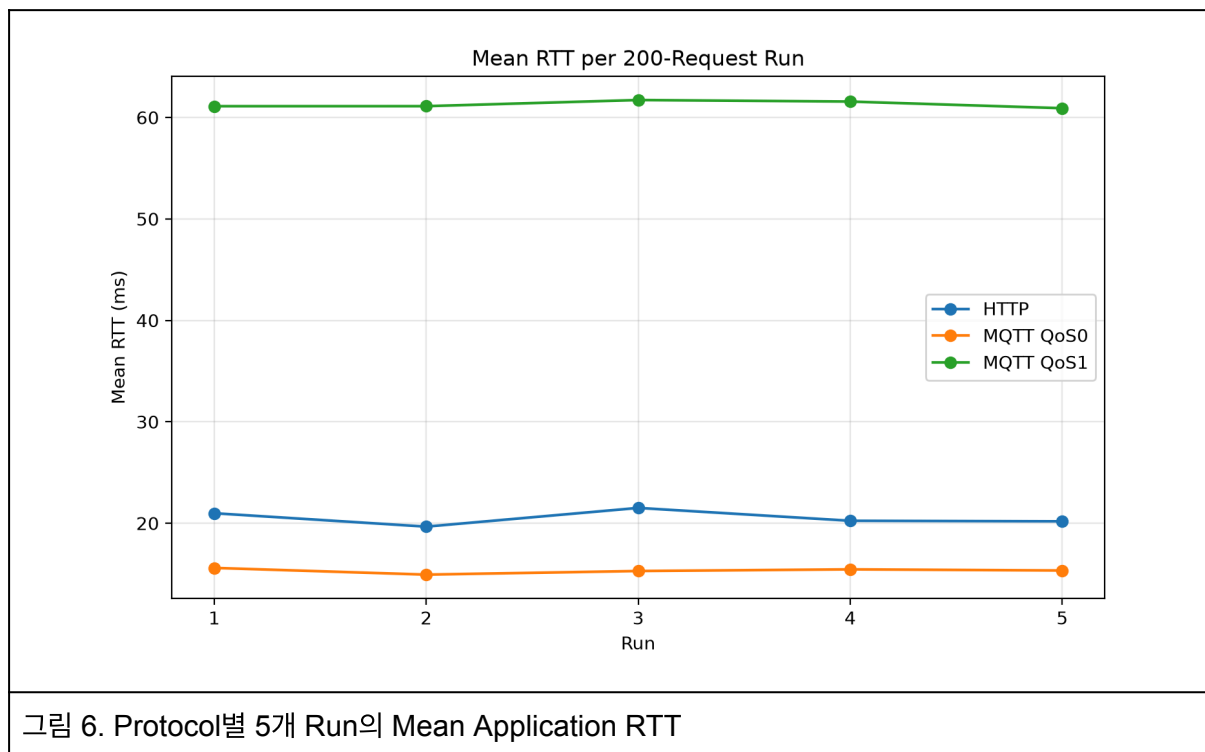
그림 5. HTTP, MQTT QoS 0, MQTT QoS 1의 Median, P95 및 P99 비교

7.3 Latency Distribution 및 Run별 재현성

Protocol별 5개 Run의 Mean은 다음과 같음.

Run	HTTP	MQTT QoS 0	MQTT QoS 1
Run 1	20.982 ms	15.592 ms	61.108 ms
Run 2	19.661 ms	14.932 ms	61.111 ms
Run 3	21.506 ms	15.286 ms	61.717 ms
Run 4	20.240 ms	15.449 ms	61.565 ms
Run 5	20.184 ms	15.338 ms	60.910 ms

Protocol Order를 Run별로 변경하였음에도 각 Protocol의 Mean은 비교적 비슷한 범위에서 반복됨. HTTP는 약 19.7~21.5 ms, MQTT QoS 0는 약 14.9~15.6 ms, MQTT QoS 1은 약 60.9~61.7 ms 범위에 위치함. 따라서 Final Mean이 특정 Run 하나의 우연한 결과에 의해 크게 결정된 것은 아니라는 점을 확인함.



Distribution을 확인하기 위해 Boxplot도 생성함. Mean뿐 아니라 Median, P95, P99에서도 Protocol별 차이가 나타남. 유효한 실험 조건에서 발생한 높은 Sample은 임의로 제거하지 않음.

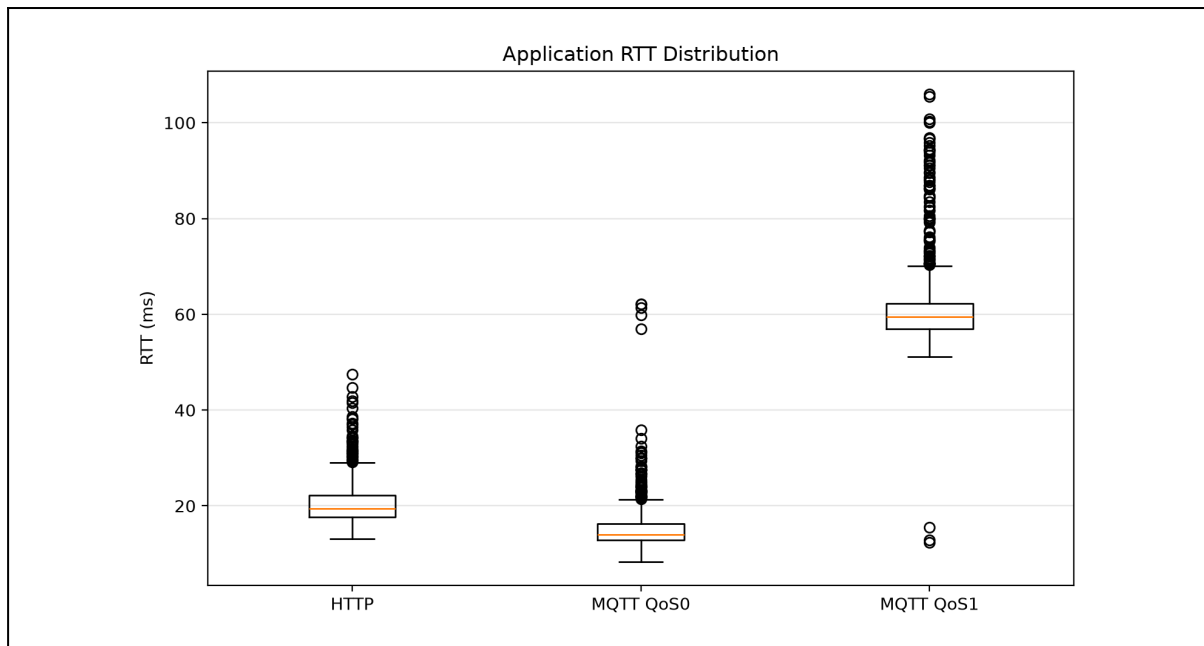


그림 7. Protocol별 Application RTT Distribution

Request Sequence Graph도 생성함. 다만 latency_sequence.png의 X-axis는 각 Protocol별로 정리한 Sample Sequence이며 전체 3000개의 Global Chronological Order를 의미하지 않음. 따라서 Sequence Graph는 각 Protocol 내부의 Sample 변화 확인 용도로 사용함.

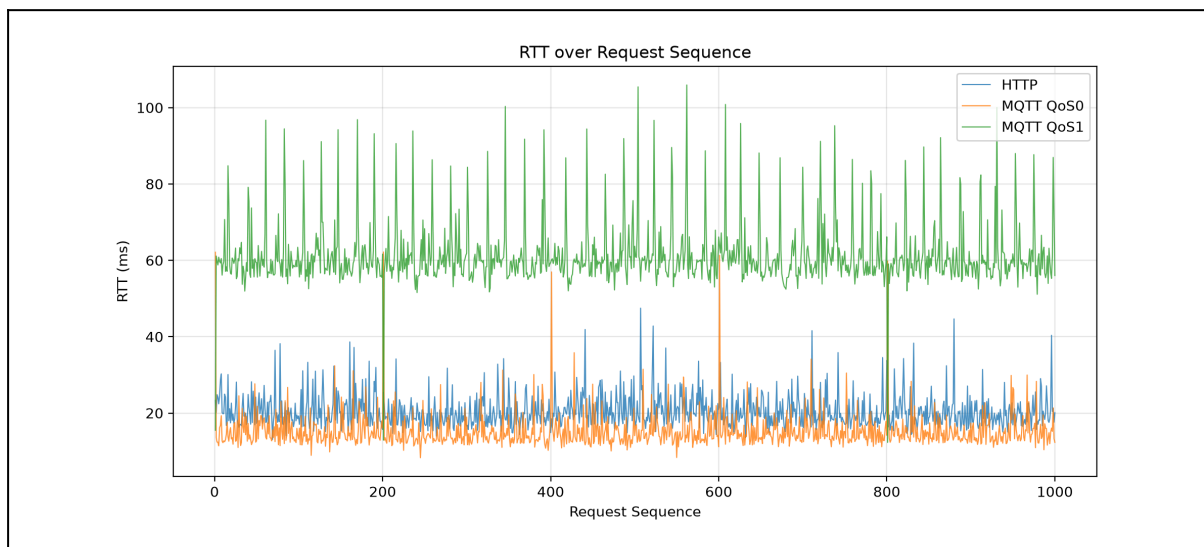


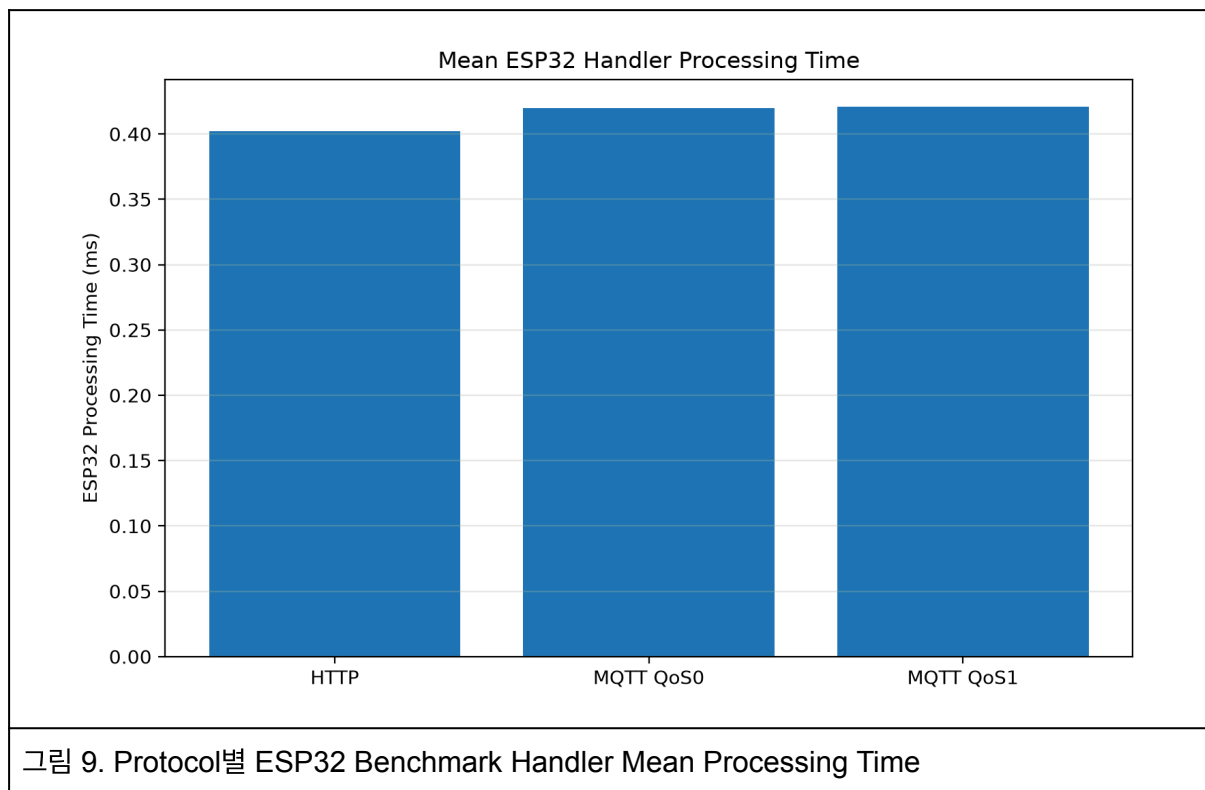
그림 8. Protocol별 Sample Sequence에 따른 Application RTT 변화

7.4 ESP32 내부 Processing Time 분석

Protocol Benchmark에서는 전체 Application RTT와 별도로 ESP32 Handler 내부 Processing Time을 측정함. 결과는 다음과 같음.

Protocol	Mean	Median	P95	Max
HTTP	401.999 μ s	390 μ s	472.05 μ s	1027 μ s
MQTT QoS 0	419.632 μ s	411 μ s	461 μ s	664 μ s
MQTT QoS 1	420.765 μ s	411 μ s	465.05 μ s	652 μ s

세 Protocol 모두 Mean 약 0.4 ms 수준으로 나타남. 반면 전체 Application RTT Mean은 15.319~61.282 ms 범위임. 이를 통해 Protocol별 전체 RTT 차이가 본 실험에서 측정한 ESP32 Benchmark Handler 내부 Processing Time에 의해 지배된 것은 아니라는 점을 확인함.



7.5 Application RTT의 구성 및 Remainder 분석

ESP32 Handler Processing과 전체 Application RTT의 크기를 비교하기 위해 다음 값을 계산함.

$$\text{Host / Network / Protocol Remainder} = \text{Application RTT} - \text{Measured ESP32 Handler Processing}$$

Mean 기준 결과는 다음과 같음.

Protocol	Application RTT	ESP Processing	Remainder
HTTP	20.515 ms	0.402 ms	20.113 ms
MQTT QoS 0	15.319 ms	0.420 ms	14.900 ms
MQTT QoS 1	61.282 ms	0.421 ms	60.862 ms

ESP32 내부에서 측정한 Processing은 세 조건에서 거의 비슷하지만 Remainder는 Protocol에 따라 큰 차이를 보임. 따라서 전체 Application RTT의 대부분은 본 실험에서 측정한 ESP32 Handler Processing 이외의 구간에 존재함.

그러나 이 Remainder에는 다음 요소가 혼합되어 있을 수 있음.

- Raspberry Pi Host Processing
- Wi-Fi Communication
- TCP 또는 MQTT Protocol Handling
- Mosquitto Broker Handling
- Response Generation
- Response Reception
- Runtime Scheduling

본 프로젝트에서는 이 요소들을 각각 독립적으로 Timestamp하지 않음. 따라서 Remainder를 순수 Network Latency라고 부르지 않음.

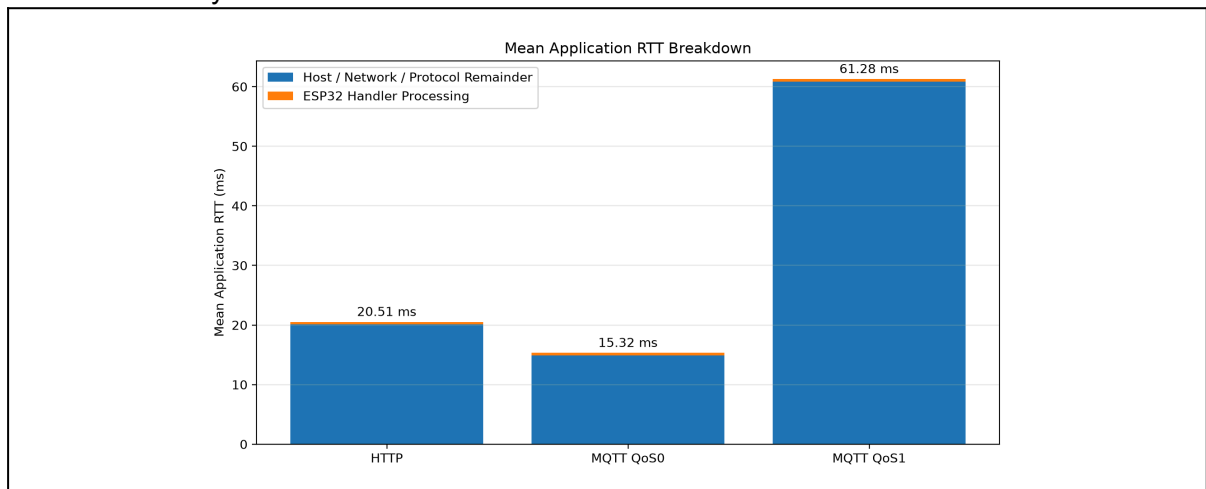


그림 10. Mean Application RTT에서 ESP32 Processing 및 Remainder가 차지하는 구성

7.6 ESP32 Heap 변화 분석

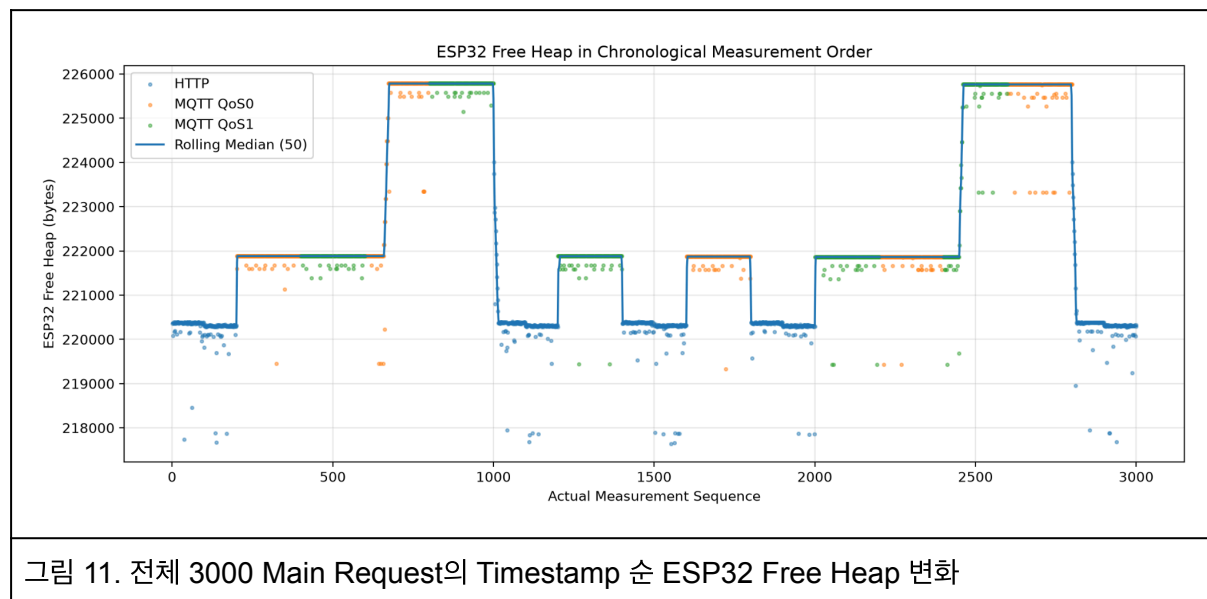
각 Main Request에서 ESP32 Heap 정보를 기록함. 처음에는 HTTP, MQTT QoS 0, MQTT QoS 1의 CSV별 First/Last Free Heap을 비교하려고 하였음.

그러나 Main Benchmark가 Protocol별 1000회를 완전히 연속해서 수행한 구조가 아니라 200회 단위로 Protocol 순서를 교차한 구조였음. 따라서 Protocol별 File의 First/Last Value는 실제 전체 Runtime의 시작과 끝을 의미하지 않음. 이 점을 확인한 후 전체 3000 Sample을 Timestamp 기준으로 다시 정렬하여 Chronological Free Heap을 분석함.

Free Heap은 약 220 KB, 222 KB, 226 KB 부근의 여러 Runtime Level 사이에서 변화하는 형태가 나타남. 다만 3000회 전체에서 시간이 지날수록 지속적으로 감소하는 형태는 관찰되지 않음. 따라서 본 결과는 다음과 같이 해석함.

“본 Benchmark 범위에서는 반복 Request에 따른 누적적인 Free Heap 감소 현상이 관찰되지 않음.” 이를 “Memory Leak이 존재하지 않음”이라고 확대하여 표현하지 않음.

또한 Minimum Free Heap은 Boot 이후 관찰된 Minimum Value이므로 각 Protocol의 순간적인 Memory Footprint를 직접 비교하는 지표로 사용하지 않음. Maximum Allocatable Heap 역시 기록하였으나 본 실험에서는 이를 기반으로 Heap Fragmentation의 원인을 확정하지 않음.



7.7 Raspberry Pi 자원 사용량 분석

Raspberry Pi Resource Benchmark의 최종 3-Run 평균 결과는 다음과 같음.

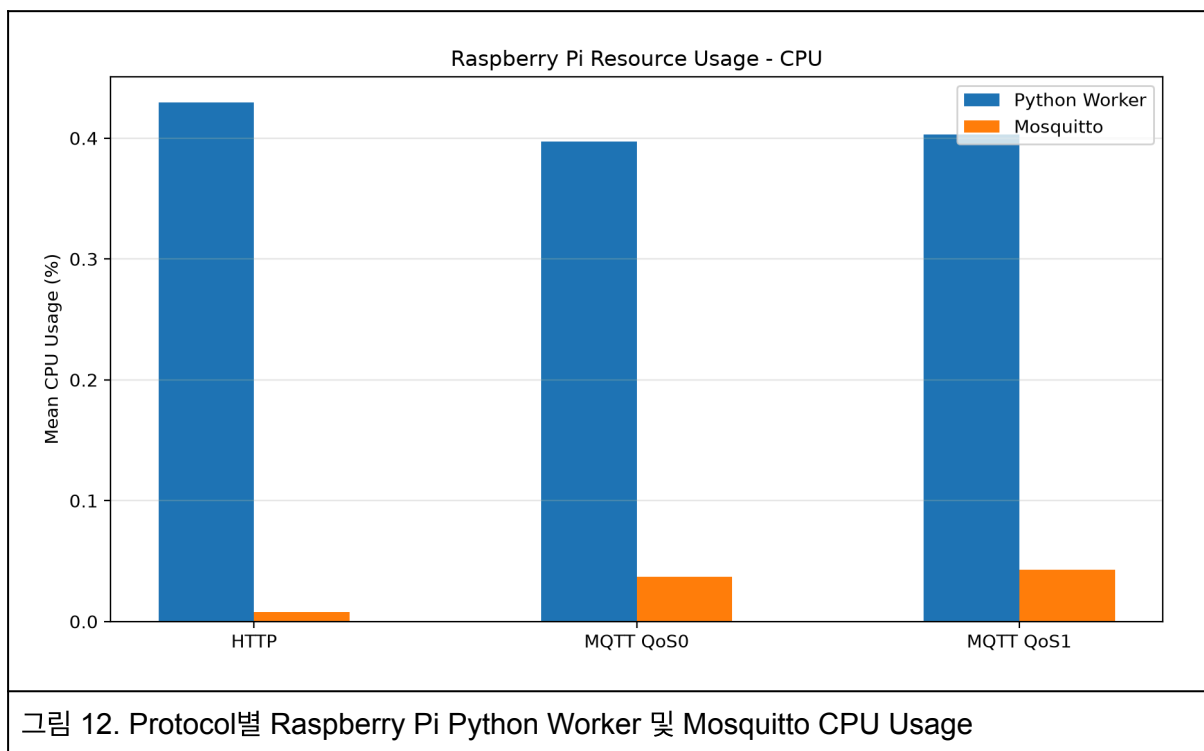
Protocol	Worker CPU	Worker RSS	Mosquitto CPU	Mosquitto RSS
HTTP	0.429%	23.165 MiB	0.008%	8.406 MiB
MQTT QoS 0	0.397%	23.371 MiB	0.037%	8.406 MiB
MQTT QoS 1	0.403%	23.389 MiB	0.043%	8.406 MiB

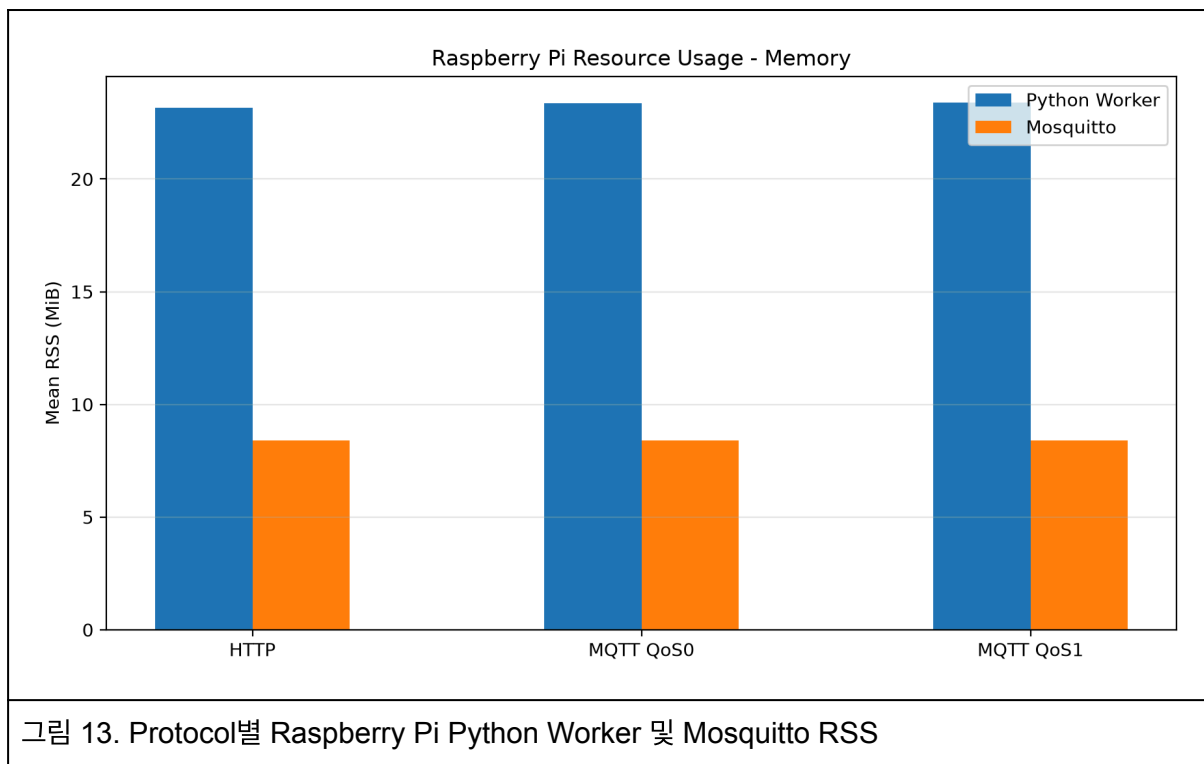
Python Worker CPU는 세 Protocol에서 약 0.4% 수준으로 나타남. Protocol간 차이는 매우 작음. MQTT 조건에서는 Mosquitto CPU Activity가 HTTP Condition보다 증가하지만 절대값은 낮은 수준임.

Python Worker RSS 역시 약 23.2~23.4 MiB 범위로 큰 차이를 보이지 않음. Mosquitto RSS는 약 8.406 MiB 수준으로 나타남.

따라서 현재의 0.2초 간격 Sequential Workload에서는 HTTP 또는 MQTT가 Raspberry Pi 5에 큰 CPU 및 Memory 부담을 주는 현상은 나타나지 않음.

다만 해당 CPU 값은 Request 사이의 0.2초 Sleep을 포함한 Workload Average임. 따라서 Request 하나의 CPU Cost 또는 Maximum Load 상황의 CPU Usage로 해석하지 않음.





7.8 Light Switch End-to-End Latency

Protocol Benchmark는 Servo Control을 의도적으로 제외하였으므로 실제 사용자가 느끼는 전체 Light Control Time을 별도로 확인할 필요가 있었음. 실제 HTTP Light Endpoint에서 ON 10회와 OFF 10회를 측정함. 결과는 다음과 같음.

Metric	Result
ON Mean	1032.498 ms
OFF Mean	1031.995 ms
Overall Mean	1032.247 ms
Overall Median	1031.450 ms
Min	1028.087 ms
Max	1040.075 ms

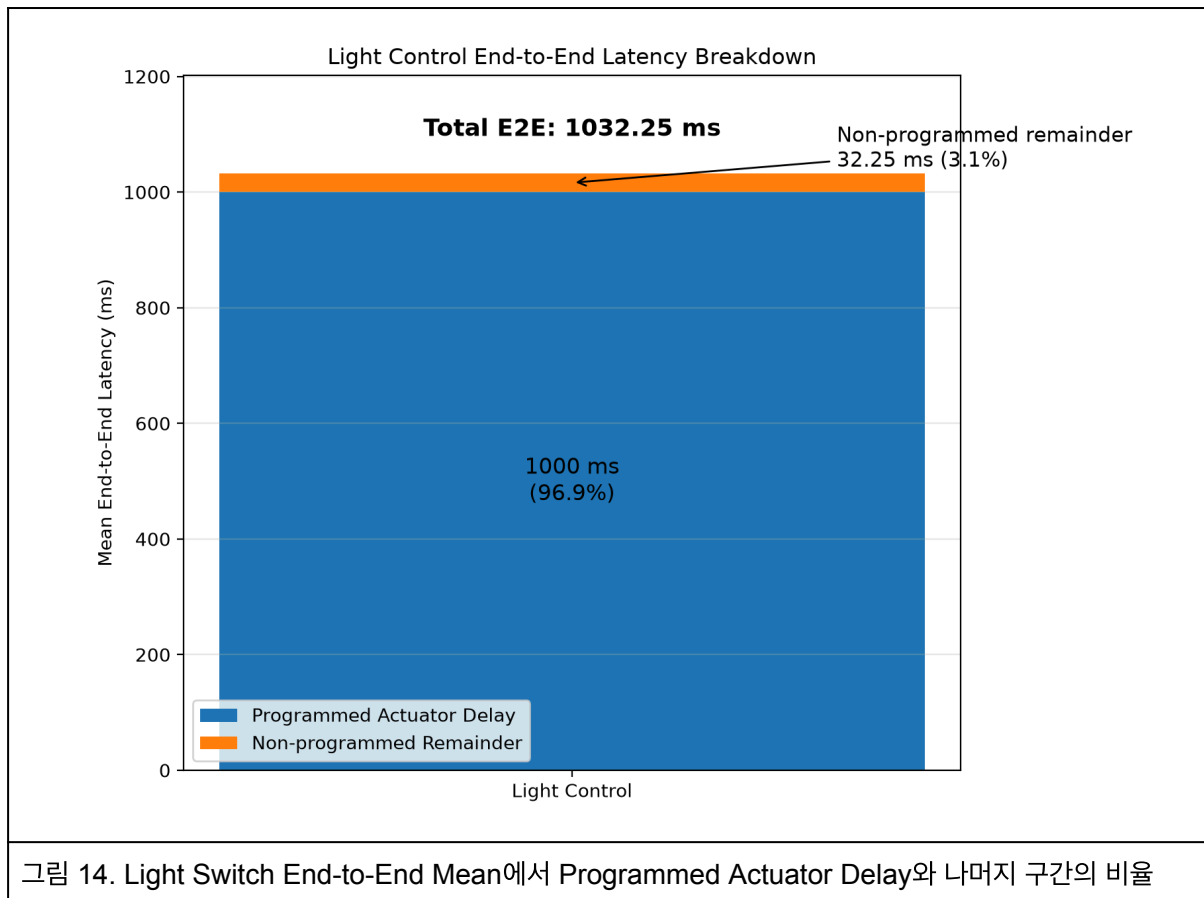
Light Firmware의 Programmed Timing은 다음과 같음.

Press Hold	→ 400 ms
Return Wait	→ 600 ms
Total Programmed Delay	→ 1000 ms

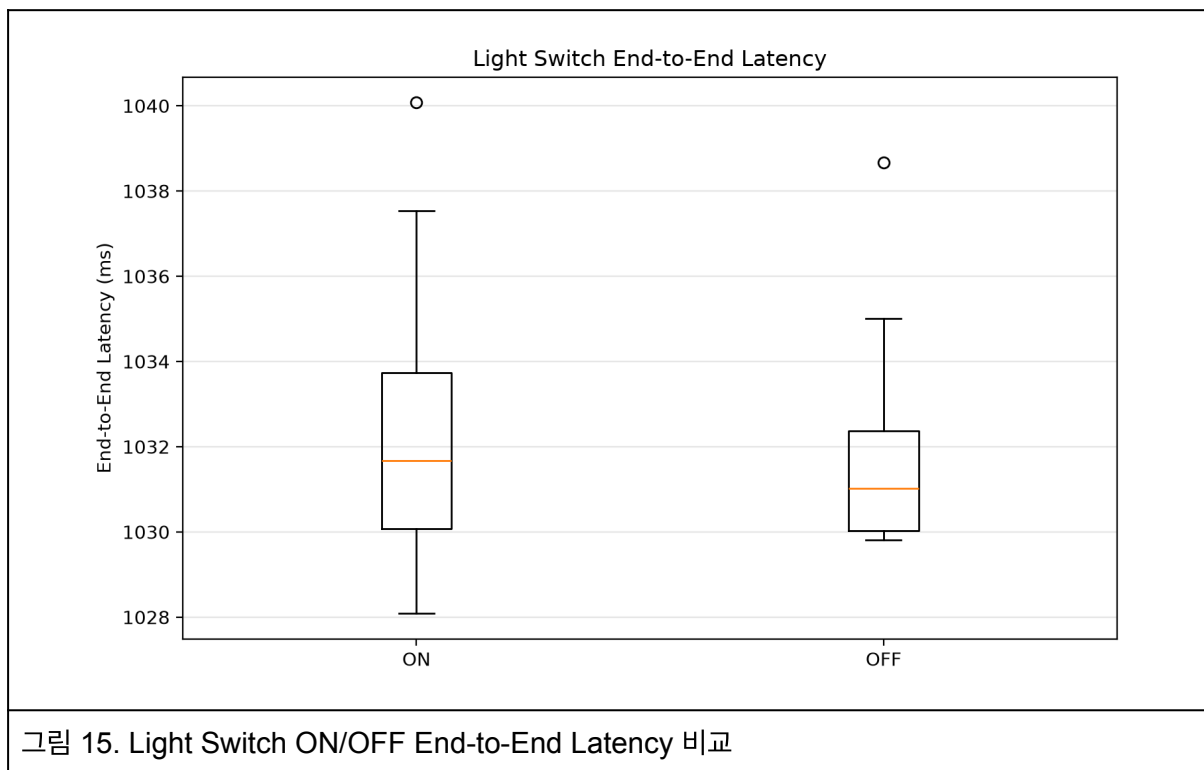
Overall Mean 1032.247 ms에서 Programmed Delay 1000 ms가 차지하는 비율은 약 96.9%임. 나머지는 약 32.247 ms로 약 3.1%임.

따라서 실제 Light Switch Control에서 사용자가 체감하는 약 1초의 Response Time은 Communication RTT보다 Programmed Servo Sequence가 훨씬 큰 비중을 차지함. 즉 Protocol Benchmark에서 15 ms와 20 ms의 차이는 Communication Layer에서는 의미가 있지만, 실제 사용자 체감 Light Control Time을 지배하는 것은 현재 Firmware의 Physical Control Timing임.

다만 32.247 ms의 Remainder 역시 순수 Network Latency는 아님. Raspberry Pi HTTP Handling, Wi-Fi/TCP/HTTP, ESP32 Handler, Servo Command 실행, Response Handling 및 Scheduling 등이 함께 포함될 수 있음.



ON과 OFF의 Mean은 약 0.5 ms 차이로 매우 유사하게 나타남. 동일한 Press Hold와 Return Wait를 사용하기 때문에 두 방향의 E2E Distribution 역시 큰 차이를 보이지 않음.



7.9 종합 Bottleneck 분석

전체 결과를 하나로 정리하면 본 시스템의 Bottleneck은 하나의 고정된 요소라고 보기 어려움. 어떤 기준으로 시스템을 평가하는지에 따라 중요한 요소가 달라짐.

Communication 관점에서는 Protocol 및 ESP32 Wi-Fi Configuration이 영향을 줌. 특히 Wi-Fi Sleep ON/OFF에 따라 MQTT QoS 0 Mean RTT가 약 121 ms와 16 ms 수준으로 크게 달라짐. 최종 동일 조건에서는 MQTT QoS 0가 15.319 ms, HTTP가 20.515 ms, MQTT QoS 1이 61.282 ms의 Mean Application RTT를 보임.

ESP32 Processing 관점에서는 세 Protocol의 Benchmark Handler Mean이 약 0.4 ms로 비슷함. 따라서 현재 Benchmark Handler 계산 자체가 Protocol별 RTT 차이의 주요 원인으로 나타나지는 않음.

Raspberry Pi Resource 관점에서는 현재 Sequential Workload에서 CPU 및 RSS가 낮은 수준임. 따라서 현재 Request Rate에서는 Raspberry Pi Resource가 주요 Bottleneck으로 확인되지 않음.

사용자 체감 Light Control 관점에서는 약 1032 ms 중 1000 ms가 Programmed Servo Delay임. 따라서 실제 User-visible Latency에서는 Physical Actuator Timing이 가장 큰 비중을 차지함.

Physical Reliability 관점에서는 초기 MG90S 조건에서 35%였던 성공률이 MG996R, Mounting 및 Angle Calibration 이후 최종 40/40으로 향상됨.

이 결과를 통해 “Bottleneck”은 하나의 Component를 의미하기보다 어떤 Metric을 기준으로 시스템을 평가하는지에 따라 다르게 나타날 수 있다는 점을 확인함.

8. 문제 해결 과정 및 결과 해석

8.1 통신 성공과 물리 제어 성공의 분리

프로젝트 초기에는 User Command가 ESP32에 전달되고 Servo가 움직이면 Control이 성공했다고 생각하기 쉬웠음. 그러나 실제 MG90S Wall Switch 시험에서는 HTTP Communication과 ESP32 Handler가 정상적으로 동작하였음에도 Physical Switch가 동작하지 않는 경우가 반복됨.

이 경험을 통해 다음 성공 조건이 서로 다를 수 있다는 점을 확인함.

- Request가 전달되었는가?
- ESP32 Handler가 정상적으로 실행되었는가?
- Servo Command가 실행되었는가?
- Servo가 실제 Switch를 충분히 눌렀는가?
- 최종 Target Device State가 변경되었는가?

따라서 이후에는 API Response Success와 Physical Actuation Success를 구분하여 기록함. Light E2E Experiment의 success 또한 HTTP API Response 성공을 의미하며 별도의 40/40 Physical Reliability Test와 동일한 값으로 해석하지 않음.

8.2 서보모터 Torque·Mounting·Angle 문제

초기 MG90S Success Rate가 35%로 나타난 뒤 가장 먼저 Servo Torque를 의심함. 실제 Wall Switch가 일정한 기계적 저항을 가지므로 더 높은 Torque의 Servo가 필요하다고 판단함.

그러나 MG996R로 변경한 직후에는 오히려 Switch를 필요 이상으로 깊게 누르는 문제가 나타남. Angle을 줄이면 접촉하지 않는 경우가 있었고, 조금 더 늘리면 접촉하지만 Switch가 완전히 전환되지 않는 경우도 있었음. Servo Mount가 충분히 고정되어 있지 않으면 Servo Horn의 Movement 일부가 Mount 자체의 움직임으로 소모되는 현상도 영향을 줌.

따라서 단순한 Servo Model 변경보다 Mounting과 Angle Calibration을 함께 수행함. 최종적으로 ON/OFF 각각 20회의 Test가 모두 성공함.

이 과정에서 실제 Physical System의 Reliability는 Component Specification 하나만으로 결정되지 않으며 Actuator, Geometry, Mounting 및 Control Parameter가 함께 영향을 준다는 점을 경험함.

8.3 예상보다 높았던 MQTT Latency 분석

MQTT QoS 0를 추가한 직후 약 100 ms 수준의 RTT가 반복적으로 나타남. 처음에는 새롭게 추가한 MQTT 자체의 특성이나 Broker Processing 때문에 높은 값이 나온 것으로 생각할 수도 있었음.

그러나 QoS 0에서도 반복적으로 큰 Delay가 나타나는 것이 이상하다고 판단하여 MQTT만 원인으로 단정하지 않고 ESP32 Network Configuration을 확인함. 그 과정에서 Wi-Fi Sleep 설정을 별도 변수로 두어 비교함.

Sleep ON 30회에서 Mean 121.279 ms가 나타났고 Sleep OFF 30회에서는 Mean 15.886 ms가 나타남. 이후 Main Benchmark에서도 MQTT QoS 0 Mean 15.319 ms가 관찰됨.

따라서 처음 관찰한 높은 RTT를 그대로 “MQTT의 성능”이라고 결론내렸다면 잘못된 비교가 될 수 있었음. Protocol 성능을 비교할 때 Device의 Power Saving 또는 Network Configuration 역시 가능한 동일하게 고정해야 한다는 점을 확인함.

8.4 HTTP Keep-Alive 실험과 제외 판단

HTTP와 MQTT는 Connection Model이 다르기 때문에 HTTP에서 Persistent Connection을 사용하면 결과가 달라질 수 있다고 생각함. 이에 Python requests.Session()을 이용한 Keep-Alive Test를 시도함. 처음에는 Session을 사용하므로 실제 TCP Connection이 재사용될 것으로 예상함.

그러나 Debug 과정에서 Server Response에 다음 Header가 확인됨.

Connection: close

Client Log에서도 dropped connection을 다시 생성하는 동작이 나타남. 즉 의도한 Persistent Connection 조건이 실제로 성립하지 않았음.

따라서 해당 결과를 “HTTP Keep-Alive Benchmark”라고 사용하면 실험 조건을 잘못 표현하게 됨. 이에 해당 Data를 최종 Protocol 비교에서 제외하고 raw/excluded/keepalive/에 별도로 보존함. 결과가 좋거나 나쁘다는 이유가 아니라 실험 조건 자체가 유효하지 않았기 때문에 제외한 것임.

이 과정에서 Benchmark를 수행하는 것만큼 실제 조건이 의도한 대로 적용되었는지를 확인하는 것도 중요하다는 점을 경험함.

8.5 Raspberry Pi Resource 실험 재설계

첫 번째 Raspberry Pi Resource Benchmark를 완료한 뒤 Worker Code를 다시 확인함. 초기 Worker는 HTTP Condition에서도 HTTP 관련 Module뿐 아니라 MQTT 관련 Module을 함께 Import하고 있었음. CPU Usage를 단순 관찰하는 데는 큰 문제가 아닐 수 있으나 Protocol별 Python Worker RSS를 비교하려는 목적에는 조건이 충분히 분리되지 않았다고 판단함.

이에 첫 번째 Resource Data를 최종 결과에 그대로 사용하지 않음. 초기 결과는 Diagnostic Data로 보존함. Worker를 수정하여 HTTP Condition에서는 HTTP 관련 Module, MQTT Condition에서는 MQTT 관련 Module이 필요한 방식으로 Import되도록 변경한 뒤 실험을 다시 수행함. 최종 Resource Result는 재실험한 Data만 이용함.

8.6 ESP32 Heap 분석 방법 수정

Protocol별 Main Data를 처음 확인했을 때 각 File의 First Free Heap과 Last Free Heap 사이에 차이가 존재함. 처음에는 이를 Protocol별 Memory Usage의 차이로 해석할 가능성이 있었음.

그러나 Main Experiment Order를 다시 확인하면 각 Protocol이 1000회씩 한 번에 연속 실행된 것이 아니었음. 200 Request 단위로 Protocol을 바꾸면서 5개 Run을 수행함. 따라서 각 Protocol File의 First/Last Sample은 전체 ESP32 Runtime의 First/Last를 의미하지 않음. 이에 기존 방식으로 Protocol별 Heap 사용량을 비교하는 것은 적절하지 않다고 판단함.

전체 HTTP, MQTT QoS 0, MQTT QoS 1의 3000 Sample을 Timestamp 기준으로 다시 정렬하고 Chronological Free Heap Graph를 생성함. 그 결과 여러 Runtime Level 사이에서 Heap이 변화하지만 전체 시간이 진행될수록 계속 감소하는 형태는 나타나지 않음.

이를 통해 Data Analysis에서도 실험값 자체뿐 아니라 실제 Experiment Execution Order를 함께 고려해야 한다는 점을 확인함.

8.7 의도된 Delay와 성능 병목의 구분

Light Servo Function에는 Press Hold 400 ms와 Return Wait 600 ms가 있음. 따라서 사용자가 약 1초를 기다린다고 해서 이 시간을 모두 Processing Overhead라고 보는 것은 부정확함.

Protocol Benchmark를 통해 실제 Communication RTT는 수십 ms 수준임을 확인함. 실제 Light E2E에서는 약 1032 ms가 측정됨. 두 결과를 함께 비교했을 때 실제 Light Control의 User-visible Latency 대부분은 Servo를 안정적으로 동작시키기 위해 의도적으로 삽입한 Control Time임을 알 수 있었음.

이 결과를 통해 성능을 분석할 때 단순히 전체 시간을 측정하는 것보다 해당 시간이 왜 존재하는지를 먼저 구분해야 한다는 점을 경험함.

8.8 측정 데이터 정합성 및 Raw Data 관리

실험이 증가하면서 Diagnostic 및 Excluded Data도 발생하였기 때문에 Raw Data Folder를 Main, Diagnostic, Excluded, Legacy로 분리함. Processed Summary와 Figure는 Raw Data를 직접 수정하여 만드는 것이 아니라 Python Analysis Script를 이용하여 별도로 생성함. 이를 통해 사람이 수기로 기록한 Summary보다 Raw Measurement와 재현 가능한 Analysis Script를 Source of Truth로 유지하는 방향으로 Data를 정리함.

8.9 결과 해석 시 주의점

본 프로젝트에서는 측정 결과를 가능한 한 실제 실험이 지원하는 범위 안에서 해석하고자 함.

첫째, MQTT QoS 0가 본 실험에서 가장 낮은 RTT를 보였지만 모든 Network 및 Implementation에서 HTTP보다 빠르다고 일반화하지 않음.

둘째, MQTT QoS 1이 높은 RTT를 보였지만 내부 Protocol Stage를 별도로 측정하지 않았으므로 어느 단계 하나가 정확한 원인이라고 단정하지 않음.

셋째, Application RTT에서 ESP32 Processing을 뺀 Remainder를 순수 Network Latency라고 표현하지 않음.

넷째, ESP32 Free Heap에서 지속적인 하락이 관찰되지 않았지만 Memory Leak이 절대 존재하지 않는다고 결론내리지 않음.

다섯째, Main Protocol Benchmark가 3000/3000 Success였지만 Packet Loss 조건을 구성하지 않았기 때문에 QoS 0와 QoS 1의 Reliability 차이를 검증했다고 해석하지 않음.

여섯째, Raspberry Pi CPU Usage는 0.2초 Request Interval을 포함한 Workload Average이므로 Per-request CPU Cost 또는 Maximum Throughput 상황으로 일반화하지 않음.

일곱째, Light E2E는 실제 Switch Contact 순간을 별도의 Sensor로 Timestamp한 값이 아니라 Application Request Completion Time임.

8.10 실험의 한계

본 프로젝트는 상용 IoT System 전체를 일반화하기 위한 대규모 Benchmark가 아님. 다음과 같은 한계를 가짐.

1. 하나의 Raspberry Pi 5와 제한된 ESP32 Node를 이용함.
2. 하나의 Local Wi-Fi 환경에서 실험함.
3. Wi-Fi Interference와 Background Traffic을 정밀한 실험 장비로 통제하지 않음.
4. HTTP와 MQTT의 Library 및 Connection Model이 서로 다름.
5. Raspberry Pi와 ESP32 Clock이 동기화되어 있지 않음.
6. Client, Raspberry Pi, Network, ESP32 전체 구간을 각각 독립적으로 Timestamp하지 않음.
7. 실제 Switch Contact 순간을 External Sensor로 Timestamp하지 않음.
8. MG90S와 MG996R 비교에서는 Servo Model뿐 아니라 Mounting과 Angle도 함께 변경됨.
9. Heap Measurement만으로 Memory Leak 또는 Fragmentation의 내부 원인을 확정할 수 없음.
10. Packet Loss와 Network Congestion 조건을 별도로 구성하지 않음.
11. Raspberry Pi Resource Benchmark는 Sequential Workload이며 Maximum Throughput Benchmark가 아님.
12. Servo Stability Test는 10분 범위의 Short-Term Experiment임.

따라서 결과는 본 프로젝트의 Hardware, Firmware, Local Network 및 Experimental Condition에서 관찰된 결과로 해석함.

9. 종합 결론

9.1 시스템 구현 결과

본 프로젝트에서는 Raspberry Pi 5와 ESP32를 이용하여 실제 생활 환경의 물리 장치를 제어하는 Edge IoT Control System을 구현함. Light Switch Node에서는 Servo를 이용하여 실제 Wall Switch를 제어함. PC Power Node에서는 Raspberry Pi의 Ping과 ESP32의 LDR 측정값을 결합하여 현재 PC State를 확인하고 필요한 경우에만 Power Button Control을 수행함.

초기 MG90S Light Switch 구성의 Success Rate는 35%였음. 이후 MG996R, Mounting 강화 및 Angle Calibration을 적용한 최종 조건에서는 ON 20회와 OFF 20회가 모두 성공함.

단순히 Network Request가 성공하는 것에서 끝나지 않고 실제 Physical Device State까지 고려하면서 Software Success와 Physical Control Success가 서로 다르다는 점을 직접 경험함.

9.2 분석 질문별 결론

Q1. HTTP와 MQTT의 Application RTT는 어떻게 달랐는가?

본 실험 조건에서 Mean RTT는 MQTT QoS 0 15.319 ms, HTTP 20.515 ms, MQTT QoS 1 61.282 ms로 나타남. Median, P95 및 P99에서도 MQTT QoS 0가 HTTP보다 낮은 값을 보였으며 MQTT QoS 1이 높은 값을 보임. 다만 해당 결과는 현재 Local Wi-Fi와 Implementation 조건에 한정함.

Q2. Protocol별 RTT 차이는 ESP32 내부 Processing으로 설명되는가?

HTTP, MQTT QoS 0, MQTT QoS 1의 ESP32 Benchmark Handler Mean Processing은 각각 약 0.402 ms, 0.420 ms, 0.421 ms임. 전체 RTT는 약 15~61 ms 수준이므로 Protocol별 차이가 본 측정구간의 ESP32 Handler Processing에 의해 발생한다고 보기 어려움.

Q3. Wi-Fi Sleep은 MQTT Latency에 영향을 주었는가?

MQTT QoS 0 Sleep ON Test의 Mean은 121.279 ms, Sleep OFF는 15.886 ms였음. Final Main MQTT QoS 0에서도 15.319 ms가 측정됨. 따라서 본 시스템에서는 Wi-Fi Power Saving Configuration이 Application RTT에 큰 영향을 줄 수 있음을 확인함.

Q4. 반복 Workload에서 ESP32 Heap은 어떻게 변화했는가?

전체 3000 Sample을 Timestamp 순으로 확인한 결과 Free Heap은 여러 Runtime Level 사이에서 변화함. 그러나 전체 실험 동안 지속적으로 감소하는 Trend는 나타나지 않음. 따라서 본 Benchmark 범위에서 누적적인 Free Heap 감소는 관찰되지 않았다고 해석함.

Q5. Raspberry Pi Resource Usage는 주요 성능 제한 요소였는가?

현재의 Sequential Workload에서는 Python Worker CPU가 약 0.4%, RSS가 약 23 MiB 수준임. Mosquitto Resource 역시 낮은 수준으로 나타남. 따라서 현재 Request Rate에서는 Raspberry Pi CPU/RSS가 주요 Bottleneck으로 나타나지 않음.

Q6. 실제 Light Control의 사용자 체감시간에서 가장 큰 비중을 차지한 것은 무엇인가?

전체 Mean E2E는 1032.247 ms임. Programmed Servo Delay가 1000 ms로 약 96.9%를 차지함. 따라서 실제 Light Control의 사용자 체감시간에서는 Physical Actuator Timing이 가장 큰 비중을 차지함.

Q7. 물리 제어의 낮은 Success Rate는 어떻게 개선되었는가?

초기 MG90S 조건에서 35%의 Success Rate가 나타남. MG996R로 교체하고 Mounting과 Angle Calibration을 함께 수행한 후 최종 40/40 Success를 기록함. 따라서 해당 문제는 Network 또는 Raspberry Pi보다 Physical Actuation Layer의 조건과 강하게 관련되어 있었음.

9.3 프로젝트를 통해 확인한 시스템 관점

프로젝트를 시작했을 때에는 가장 중요한 질문이 “원하는 기능이 동작하는가?”였음. 하지만 실제 Hardware를 구현하고 여러 Experiment를 수행하면서 다음과 같은 질문이 중요하다는 것을 경험함.

- Request Success는 정확히 어느 단계의 성공을 의미하는가?
- 실제 Device가 원하는 State로 변경되었는가?
- 전체 시간이 길다면 왜 시간이 발생하는가?
- Programmed Delay와 Processing Overhead를 어떻게 구분할 것인가?
- 평균값 이외에 Tail Latency와 Run별 변화는 어떠한가?
- 예상과 다른 결과가 나타나면 어떤 변수를 의심해야 하는가?
- 내가 의도한 Experimental Condition이 실제로 적용되었는가?
- Resource Value의 변화가 실제 Execution Order와 어떻게 연결되는가?
- 직접 측정한 결과와 추정된 영역을 어떻게 구분해야 하는가?

특히 MQTT의 초기 높은 Latency를 그대로 Protocol 특성으로 받아들이지 않고 Wi-Fi Sleep을 별도의 변수로 확인한 과정, Keep-Alive 조건이 실제로 성립하지 않음을 확인하고 Data를 제외한 과정, Resource Experiment의 비교 조건 문제를 발견하여 다시 측정한 과정, Heap Data를 실제 Timestamp 순서로 다시 분석한 과정이 프로젝트 후반부의 중요한 경험이었음. 이를 통해 단순히 많은 기능을 구현하는 것과 이미 구현한 시스템을 측정하고 이해하는 것은 서로 다른 작업이라는 점을 알게 됨.

9.4 프로젝트의 한계

본 프로젝트는 학부 과정 중 개인적으로 수행한 Project이며 전문적인 연구용 Network Measurement 장비나 Hardware Performance Counter를 이용한 정밀 System Profiling을 목표로 한 것은 아님.

HTTP/MQTT 결과는 특정 Hardware와 Local Network 및 Library Configuration에서 수행된 결과임. ESP32 Heap 측정 역시 Embedded Runtime Resource를 관찰한 수준이며 Memory Hierarchy 자체를 분석한 것은 아님. 전체 Latency Decomposition Model의 모든 Component를 각각 독립적으로 측정하지 못하였다는 한계도 존재함.

그럼에도 실제 System을 구현한 뒤 Measurement Point를 추가하고, Raw Data를 수집하며, 문제의 원인을 조건별로 확인하고, 결과를 직접 측정한 범위 내에서 해석하려고 했다는 점에 본 프로젝트의 학습 의의가 있음.

9.5 이후 관심 방향

이번 프로젝트를 통해 단순한 기능 구현보다 실제 시스템에서 발생하는 Latency와 Resource Usage를 측정하고 원인을 분석하는 과정에 흥미를 느꼈음. 앞으로는 운영체제, 메모리 시스템, 컴퓨터 구조와 관련된 내용을 더 공부하면서 시스템 성능을 분석하는 경험을 확장하고 싶음.

10. 셀프 피드백

10.1 프로젝트를 통해 학습한 내용

이번 프로젝트를 통해 Raspberry Pi와 ESP32 사용법뿐 아니라 실제 시스템을 구성하는 여러 요소가 서로 다른 방식으로 전체 동작에 영향을 준다는 점을 경험함.

초기에는 Flask Server가 실행되고 ESP32가 Request를 받아 Servo가 움직이면 대부분의 기능이 완료된 것으로 생각함. 그러나 실제 Wall Switch를 연결한 이후에는 Servo Torque, Mounting, Angle과 같이 Code 밖의 Physical Condition이 전체 Reliability에 큰 영향을 미침.

PC Power Control에서는 Ping 하나만 이용하는 대신 LDR라는 다른 성격의 Physical Signal을 함께 사용하면서 실제 State를 확인한 뒤 Control하는 방식에 대해 생각하게 됨.

성능 측정 단계에서는 Mean Latency 하나를 확인하는 것에서 시작하였지만 이후 Median, P95, P99, Run 간 Variation, ESP32 Processing, Heap 및 Raspberry Pi Resource까지 확인하게 됨. 무엇보다 결과가 예상과 다를 경우 값을 그대로 받아들이기보다 Experimental Condition을 다시 확인하는 과정이 중요하다는 점을 경험함.

10.2 초기 프로젝트 진행 방식의 아쉬운 점

프로젝트 초기에는 구현하고 싶은 기능 자체에 집중하여 Measurement Plan을 처음부터 충분히 설계하지 못함. 기능이 어느 정도 완성된 이후에 Latency와 Resource를 측정하기 시작했기 때문에 Benchmark용 Endpoint와 Instrumentation을 나중에 별도로 추가해야 했음.

초기 HTTP 기록에서 Maximum Latency 값이 문서 사이에서 잘못 기록된 문제도 있었음. Resource Benchmark에서도 첫 번째 Experiment 이후 비교 조건을 다시 검토하여 재측정해야 했음. Heap Analysis 역시 최종 Main Run Order를 충분히 고려하지 않은 First/Last 비교에서 시작하였다가 이후 Chronological Analysis로 변경함.

이러한 시행착오를 통해 향후 유사한 프로젝트에서는 구현 전에 다음 항목을 조금 더 일찍 정의하는 것이 필요하다고 생각함.

- 무엇을 비교할 것인가?
- 성공을 어떤 기준으로 정의할 것인가?
- 어느 구간의 시간을 측정할 것인가?
- 어떤 Data를 Raw Data로 보존할 것인가?
- 비교 조건에서 반드시 동일하게 유지해야 하는 변수는 무엇인가?
- 결과가 예상과 다를 때 어떤 Diagnostic을 수행할 수 있는가?

10.3 기능 구현에서 측정 및 분석으로의 관점 변화

프로젝트 후반에는 새로운 기능을 계속 추가하기보다 이미 구현한 시스템을 하나의 실험 대상으로 바라보기 시작함. 처음에는 실제 방의 Switch가 움직이는 것 자체가 재미있었지만 이후에는 다음 질문을 확인하는 과정에 더 관심이 생김.

- 왜 Physical Switch가 실패하는가?
- 왜 MQTT QoS 0가 처음에는 약 100 ms가 나오는가?
- 왜 Wi-Fi Sleep을 끄면 결과가 크게 달라지는가?
- Protocol별 전체 RTT가 다른데 ESP32 내부 Handler는 왜 거의 같은가?
- Heap Value가 달라졌다는 사실을 실제 Runtime 순서에서는 어떻게 해석해야 하는가?
- 실제 사용자가 1초를 기다리는데 Communication RTT는 왜 수십 ms에 불과한가?

이와 같은 질문을 하나씩 확인하면서 기능을 구현하는 것과 구현한 System의 동작 원인을 분석하는 것이 서로 다른 작업이라는 점을 경험함. 특히 최종 결과값 자체보다 결과가 예상과 다를 때 가설을 세우고, 조건을 바꾸어 다시 확인하며, 잘못된 분석 방법을 수정하는 과정이 프로젝트에서 가장 의미 있었던 부분이라고 생각함.

11. 참고자료

1. Raspberry Pi Documentation
2. Espressif ESP32 Documentation
3. ESP-IDF Heap Memory 관련 Documentation
4. Arduino ESP32 Documentation
5. Python time Module Documentation
6. Flask Documentation
7. OASIS MQTT Specification
8. Eclipse Mosquitto Documentation
9. MQTT by Joel Gaehwiler Library Documentation
10. 프로젝트 GitHub Repository의 Source Code, Raw Data 및 Analysis Script

부록 A. 주요 실험 데이터 및 Figure

A.1 Final Protocol Raw Data

data/raw/main/http/
http_main_1000_nosleep_20260904_102455.csv

data/raw/main/mqtt_qos0/
mqtt_qos0_main_1000_nosleep_20260904_102455.csv

data/raw/main/mqtt_qos1/
mqtt_qos1_main_1000_nosleep_20260904_102455.csv

A.2 Raspberry Pi Resource Data

data/raw/main/resource/
pi_resource_samples_20260906_002151.csv

data/processed/
pi_resource_summary_20260906_002151.csv

A.3 Light End-to-End Data

data/raw/main/end_to_end/
light_http_e2e_20_20260906_010354.csv

data/processed/
light_e2e_summary.csv

A.4 주요 Figure

data/figures/latency_percentiles.png
data/figures/per_run_mean_latency.png
data/figures/latency_boxplot.png
data/figures/latency_sequence.png
data/figures/esp_processing_mean.png
data/figures/latency_breakdown.png
data/figures/free_heap_chronological.png
data/figures/pi_resource_cpu.png
data/figures/pi_resource_rss.png
data/figures/light_e2e_breakdown.png
data/figures/light_e2e_on_off.png

부록 B. 실험 재현을 위한 환경 기록

실험을 다시 수행하거나 결과를 비교할 경우 가능한 다음 정보를 함께 기록함.

- Git Commit Hash
- Raspberry Pi OS Version
- Python Version
- Python Package Version
- Arduino IDE Version
- ESP32 Board / Core Version
- MQTT Library Version
- Mosquitto Version
- Raspberry Pi IP
- ESP32 IP
- Wi-Fi Sleep Condition
- RSSI
- Servo Model
- Servo REST / ON / OFF Angle
- Press Hold
- Return Wait
- LDR Threshold
- HTTP Benchmark Endpoint
- MQTT Topic
- Warm-up Count
- Main Sample Count
- Run Order
- Request Interval
- Timeout